Adaptive dynamic query routing for Urdu information retrieval

Hashim Shazad[1*], Adnan Aslam[1]

1 Department of Creative Technologies, Air University, Islamabad, Pakistan

* abbasihashim30@gmail.com

Abstract

Urdu information retrieval (IR) systems face persistent challenges from script variability, morphological complexity, and the widespread use of Roman Urdu in user queries, yet existing adaptive retrieval frameworks such as ULTRA rely on a single static, length-based routing threshold that does not adapt to a query's deeper linguistic characteristics. This paper proposes a dynamic query routing extension to the ULTRA framework, replacing its static threshold with a lightweight, eight-feature Support Vector Machine (SVM) classifier combined with a three-tier confidence-based escalation strategy. Queries are additionally processed through an expanded Roman Urdu transliteration dictionary (grown from 30 to 179 entries) to improve retrieval precision on code-mixed and transliterated text. Evaluated on a dataset of 369 real Urdu queries, the proposed dynamic routing framework achieves 100% routing accuracy (AUC = 1.000), compared to 50% for the static threshold-based baseline, with an average routing confidence of 98.18% and a Roman Urdu retrieval precision (P@15) of 92.5% – raising overall system P@15 from 43.75% (static ULTRA, which cannot process Roman Urdu at all) to 90.00%. Comparison against four LLM-based routing baselines shows the proposed classifier matches their routing accuracy at approximately three orders of magnitude lower per-query latency (0.4555 ms vs. 600–900 ms) and zero API cost. Robustness validation – including effect size analysis (Cohen's d up to 14.2), learning curve inspection, regularization sensitivity testing, and evaluation on 50 held-out unseen queries (100% accuracy) – confirms these improvements generalize beyond the training distribution. These results demonstrate that dynamic, confidence-aware query routing offers a substantial and reliable improvement over static routing for Urdu information retrieval.

Introduction

Background

Urdu is spoken by over 230 million people worldwide and serves as the national language of Pakistan and one of the officially recognized languages of India, yet it remains substantially underserved within contemporary information retrieval (IR) research [1]. Unlike high-resource languages such as English, where dense retrieval, neural ranking, and large language model (LLM)-based query understanding have matured rapidly – driven by pretrained transformer encoders [2] and dense dual-encoder retrievers [3] – Urdu IR continues to face persistent challenges rooted in script complexity, morphological richness, and the informal, code-mixed nature of user-generated queries [1, 4]. A particularly acute challenge is the widespread use of Roman Urdu – Urdu written using the Latin script – in everyday digital communication and search queries, which introduces significant lexical and orthographic variation that conventional retrieval pipelines are not designed to handle [5].

The scarcity of standardized evaluation resources compounds this problem. Early efforts such as the Collection for Urdu Retrieval Evaluation (CURE) [6] and, more recently, the translated Urdu MS MARCO baseline [7], have begun to establish benchmarks for Urdu IR, but adaptive, query-aware retrieval strategies for Urdu remain largely unexplored.

The ULTRA framework, proposed by Bashir, Qaiser, and Hussain [8], represents a recent effort in this direction, introducing a dual-embedding, query-length-sensitive routing scheme that distinguishes between short, intent-oriented queries and longer, context-rich queries using a threshold-based decision process. However, ULTRA's routing mechanism relies on a *static* query-length threshold, which does not adapt to the full range of linguistic characteristics a real Urdu query may exhibit – including lexical diversity, Roman Urdu transliteration, and question structure. This mirrors a broader challenge identified in the general IR literature: Arabzadeh et al. [9] showed that a learned classifier for routing queries between sparse and dense retrieval strategies can substantially outperform static rules, though their work was developed and evaluated only for English.

This paper extends the ULTRA framework by replacing its static, length-based routing threshold with an adaptive, dynamic query routing mechanism. The proposed system classifies incoming Urdu (and Roman Urdu) queries using a lightweight, eight-feature Support Vector Machine (SVM) classifier and routes them to the most appropriate retrieval configuration in real time, combined with a confidence-based tiered escalation strategy for ambiguous queries.

Problem statement

Existing Urdu information retrieval systems, including the ULTRA framework on which this paper builds, predominantly employ static, threshold-based routing that applies a fixed decision rule regardless of the deeper linguistic characteristics of a query – such as lexical diversity, script type (native Urdu vs. Roman Urdu), or the presence of interrogative structure. This static approach results in suboptimal retrieval performance for queries that deviate from the assumption the threshold was tuned for, particularly Roman Urdu queries, which suffer from inconsistent transliteration and vocabulary sparsity [5]. Preliminary evaluation of static threshold-based routing on a diverse Urdu query set shows accuracy as low as 50%, representing a critical reliability gap for real-world deployment.

Research questions

RQ1: Can query-level structural and linguistic features (e.g., character length, lexical diversity, Urdu character ratio, presence of question words) reliably predict the optimal retrieval configuration for a given Urdu query?

RQ2: Does a dynamic, machine learning-based query routing mechanism significantly outperform static threshold-based routing, of the kind used in the base ULTRA framework [8], in terms of retrieval precision (P@15) across both native Urdu and Roman Urdu queries?

RQ3: Can a confidence-based tiered routing strategy further improve system reliability and efficiency by selectively escalating low-confidence queries to more computationally intensive retrieval methods while resolving high-confidence queries through a lightweight classifier?

Objectives

- **Dynamic query classification:** Design and train a dynamic SVM-based classifier that predicts optimal query routing decisions using eight structural and linguistic features extracted from Urdu and Roman Urdu queries.
- **Roman Urdu support:** Develop and expand a Roman Urdu transliteration dictionary (from 30 to 179 words) to improve retrieval precision for code-mixed and transliterated queries.
- **Confidence-based tiered routing:** Implement a three-tier confidence routing mechanism balancing retrieval accuracy with computational efficiency.
- **Comparative evaluation:** Benchmark the proposed dynamic routing approach against static threshold-based routing and commercial LLM-based routing methods across a dataset of 369 real Urdu queries spanning 15+ topics.
- **Robustness validation:** Statistically validate the significance and generalizability of performance improvements through effect size analysis, learning curve evaluation, and validation on unseen queries.

Contributions

1. A dynamic SVM-based query routing model achieving 100% routing accuracy on the evaluation dataset, compared to 50% for the static threshold-based routing used in the base ULTRA framework [8].
2. An eight-feature semantic classifier engineered specifically for Urdu query characterization.
3. A confidence-based three-tier routing architecture achieving an average confidence score of 98.18% across the evaluation set.
4. An expanded Roman Urdu transliteration dictionary (30 $\rightarrow$ 179 words, a 6$\times$ growth), improving Roman Urdu retrieval precision to 92.5% (P@15).
5. A comprehensive comparative evaluation against six retrieval methods, including large language models.
6. An ablation study identifying the most robust and discriminative features for Urdu query routing.

Related work

Urdu information retrieval sits at the intersection of three research threads that are individually well studied for high-resource languages but remain fragmented for Urdu: general Urdu natural language processing, information retrieval benchmarking, and query-adaptive routing. This section reviews representative work in each thread and identifies the specific gap this paper addresses.

Urdu natural language processing

Daud et al. [1] conducted a comprehensive survey of Urdu language processing, cataloguing the linguistic characteristics that complicate computational treatment of the language: a cursive, context-sensitive Perso-Arabic script written right-to-left, rich morphology, and the absence of large-scale standardized corpora compared to English or Arabic. Their survey remains a foundational reference for understanding why Urdu NLP tools lag behind those available for high-resource languages. **Research gap:** The survey catalogues foundational challenges but predates the transformer era and therefore does not address embedding-based retrieval or query-adaptive systems. **Advantages:** Comprehensive coverage of

Urdu-specific linguistic obstacles; widely cited foundational reference. **Disadvantages:** No treatment of neural retrieval, query routing, or Roman Urdu-specific processing.

Kazi and Khoja [4] more recently proposed a framework for Urdu language document retrieval, applying deep learning techniques and evaluating the effect of different word embeddings on retrieval-adjacent tasks such as named entity recognition. **Research gap:** The framework focuses on document-level representation quality rather than on how individual queries should be routed to different retrieval strategies based on their structural characteristics. **Advantages:** Recent, transformer-era treatment of Urdu retrieval; empirically evaluates multiple embedding configurations. **Disadvantages:** Does not address query-level adaptivity or Roman Urdu queries specifically.

Roman Urdu processing challenges

Roman Urdu – Urdu transliterated into the Latin script – lacks a standardized spelling convention, meaning the same word may be written in multiple ways depending on the writer's phonetic intuition. Hussain et al. [5] addressed a related problem, fine-tuning large language models via QLoRA for offensive language detection in Roman Urdu-English code-mixed text, and explicitly identified inconsistent spelling, unstated grammar, and scarcity of labeled data as core obstacles for any Roman Urdu NLP task. **Research gap:** Their work targets classification (offensive vs. non-offensive) rather than retrieval, and does not address how Roman Urdu queries should be routed differently from native-script Urdu queries within an IR pipeline. **Advantages:** Demonstrates that parameter-efficient fine-tuning (QLoRA) can meaningfully improve Roman Urdu-English task performance despite data scarcity. **Disadvantages:** Single-task focus (offensive language detection); does not generalize to query routing or retrieval precision.

More broadly, Sitaram et al. [10] surveyed code-switched speech and language processing, characterizing the alternation between languages and scripts – of which Roman Urdu is a script-level instance – as a communicative phenomenon that conventional monolingual NLP pipelines are not designed to handle. Within Urdu specifically, Mehmood et al. [11] tackled the closely related task of Roman Urdu sentiment analysis, proposing a hybrid multi-channel architecture over custom Word2Vec, FastText, and GloVe embeddings and reporting F1-score gains of up to 9% over adapted machine-learning baselines. **Research gap:** Both works confirm that Roman Urdu requires script-aware, dedicated processing rather than being treated as a minor variant of native-script Urdu, but neither addresses query routing or retrieval: the code-switching survey is a general characterization of the problem space, and the sentiment hybrid targets classification rather than document retrieval. **Advantages:** The survey provides a broad taxonomy of code-switching challenges applicable beyond Urdu; the sentiment hybrid empirically demonstrates that Roman Urdu-specific embeddings outperform generic ones on a downstream task. **Disadvantages:** Neither considers retrieval precision, confidence-based routing, or the query-length and script-composition signals used in this paper's classifier.

Low-resource and cross-lingual retrieval limitations

Urdu's challenges are not unique; a growing body of work shows that state-of-the-art dense retrieval degrades sharply once a language falls outside the small set of high-resource languages that dominate pretraining corpora. Wu et al. [12] evaluated the multilingual Dense Passage Retriever (mDPR) – the retrieval component of the Cross-lingual Open-Retrieval Answer Generation (CORA) pipeline – on two extremely low-resource languages, Amharic and Khmer, and found that even after targeted extensions using translation language modelling and question–passage alignment, retrieval quality improvements were modest and absolute performance remained low. Their finding that language alignment techniques only partially close the gap for low-resource languages is directly relevant to Urdu: general-purpose multilingual dense retrievers cannot be assumed to transfer well without language-specific adaptation, reinforcing the motivation for a dedicated, Urdu-tailored routing and retrieval pipeline rather than an out-of-the-box multilingual model. **Research gap:** Evaluated only on Amharic and Khmer within a cross-lingual (not monolingual) QA setting; does not address query routing, script-mixing, or a Latin-script transliterated variant analogous to Roman Urdu. **Advantages:** Rigorous empirical demonstration of where and why multilingual dense retrievers fail for low-resource languages; proposes concrete (if only partially effective) mitigation strategies.

Disadvantages: Improvements remain modest even after adaptation; does not consider per-query adaptive strategy selection as a complementary solution.

Closer to the transliteration problem this paper addresses, Chari et al. [13] observed that current search systems are not usually trained on low-resource language varieties, forcing users to express queries in a high-resource language instead – a pattern directly analogous to Urdu speakers defaulting to Roman (Latin-script) transliteration when native-script input is inconvenient. They proposed fine-tuning neural rankers on pairs of related language varieties so that the ranker learns to exploit shared lexical and grammatical structure, improving retrieval effectiveness for the trained varieties and generalizing, with mixed but promising results, to unseen variety pairs. **Research gap:** Targets script-consistent language varieties (e.g., Occitan, Sicilian) rather than a single language expressed in two different scripts (native Urdu vs. Roman Urdu); does not incorporate a query-level routing decision. **Advantages:** Demonstrates that explicit exploitation of linguistic similarity between variety pairs is more effective than naive multilingual pretraining alone; open-source implementation available. **Disadvantages:** Requires paired training data across varieties; cross-family transfer results are inconsistent, suggesting the technique alone would not fully resolve Roman Urdu's spelling inconsistency problem without a complementary dictionary-based approach such as the one used in this paper.

The Urdu-specific evidence within this broader literature is itself instructive: Conneau et al. [14] showed that scaling multilingual pretraining (XLM-R) improves Urdu XNLI accuracy by 11.4 points over prior multilingual models, and that low-resource languages such as Urdu benefit disproportionately from scale – yet this gain is realized at the level of a single monolithic embedding model applied uniformly to every query, with no mechanism for detecting when a given query's script, length, or lexical structure would be better served by a different retrieval strategy altogether. **Research gap:** Demonstrates that general-purpose multilingual embeddings can be substantially improved for Urdu through pretraining scale, but improvement is applied uniformly across all queries rather than adaptively; provides no per-query routing or confidence mechanism. **Advantages:** Direct, quantified evidence that Urdu specifically benefits from multilingual representation scaling; widely adopted and freely available pretrained model. **Disadvantages:** Improves embedding quality in aggregate rather than routing decisions; does not address Roman Urdu transliteration or query-level adaptivity, which this paper's dictionary-expansion and dynamic-routing components target directly.

Together, these two studies confirm a consistent pattern across unrelated low-resource settings: general-purpose neural retrieval degrades for under-represented languages and scripts, and closing the gap requires language- or script-specific engineering rather than relying on multilingual pretraining alone. This motivates the explicit Roman Urdu detection and dictionary-transliteration module used in this paper's methodology, rather than depending on the embedding model alone to bridge script variation.

Information retrieval benchmarking for Urdu

Iqbal et al. [6] introduced CURE (Collection for Urdu Retrieval Evaluation), a standard test collection of Urdu documents built specifically to enable rigorous IR evaluation, addressing the absence of a shared benchmark comparable to TREC-style collections available for English. **Research gap:** CURE provides an evaluation resource but does not itself propose an adaptive retrieval or routing mechanism – it evaluates retrieval quality, not routing decisions. **Advantages:** Enables reproducible, standardized comparison of Urdu IR systems. **Disadvantages:** Limited in scale (relatively small query set) and does not include Roman Urdu queries.

Butt et al. [7] established the first large-scale Urdu IR dataset by machine-translating the MS MARCO passage ranking dataset into Urdu, and benchmarked both BM25 and fine-tuned multilingual transformer (mT5-based mMARCO) baselines, reporting a Mean Reciprocal Rank (MRR@10) of 0.247 for their best fine-tuned configuration. **Research gap:** Their evaluation applies a single retrieval strategy uniformly to all queries; no mechanism is proposed for detecting which queries would benefit from a different retrieval configuration. **Advantages:** Large-scale dataset (translated from MS MARCO); establishes strong zero-shot and fine-tuned baselines for future Urdu IR work. **Disadvantages:** Relies on machine-translated data, introducing potential semantic misalignment; no query-adaptive component.

Query-adaptive retrieval routing

Outside the Urdu-specific literature, query routing between retrieval strategies has been studied more extensively for English. The underlying premise – that queries differ systematically in how difficult they are to satisfy, and that this difficulty can be estimated before or during retrieval – was formalized early by Carmel and Yom-Tov [15] in their synthesis of query performance prediction (QPP) methods, which surveyed pre- and post-retrieval signals for anticipating when a query is likely to fail. **Research gap:** QPP methods estimate *how well* a fixed retrieval strategy will perform on a given query, but classical QPP does not itself select *among* alternative retrieval strategies or scripts, and was not developed with Urdu or code-mixed queries in mind. **Advantages:** Established the foundational concept that per-query difficulty is estimable and actionable, underpinning all subsequent per-query routing work including this paper's confidence-based tiering. **Disadvantages:** Predates neural retrieval and transformer-based confidence estimation; no treatment of script variation, transliteration, or routing decisions between qualitatively different pipelines.

Arabzadeh et al. [9] proposed a classifier that selects between sparse (BM25), dense, and hybrid retrieval strategies on a per-query basis, training a BERT-based model to predict which strategy would be most effective for a given query before retrieval is performed. Their approach reduced computational cost (fewer calls to GPU-bound dense retrievers) while maintaining retrieval effectiveness, by routing "easy" queries to the cheaper sparse retriever and reserving the dense retriever for queries it is more likely to help. **Research gap:** This routing framework was designed and evaluated exclusively on English MS MARCO data; it does not address Urdu script variation, Roman Urdu transliteration, or the confidence-tiered escalation needed when a routing decision itself is uncertain. **Advantages:** Demonstrates, in a high-resource language, that a learned per-query routing classifier meaningfully outperforms fixed retrieval strategies; validates the core intuition this paper extends to Urdu. **Disadvantages:** Language-specific to English; binary/ternary routing decision only, with no confidence-based escalation tier; no treatment of low-resource or code-mixed scripts.

Closer to the confidence-tiered design adopted in this paper, and building on the retrieval-augmented generation paradigm introduced by Lewis et al. [16] – in which a parametric generator is conditioned on passages retrieved from a non-parametric index rather than relying on retrieval alone – Jeong et al. [17] proposed Adaptive-RAG, a framework in which a lightweight classifier – trained on automatically collected complexity labels – predicts how complex an incoming question is and routes it to one of three retrieval-augmented generation strategies: no retrieval for the simplest queries, single-step retrieval for moderately complex queries, and iterative multi-step retrieval for the most complex queries. This three-way, classifier-driven escalation strategy is structurally analogous to the three-tier (HIGH/MEDIUM/LOW confidence) routing mechanism proposed in this paper, though Adaptive-RAG operationalizes complexity as a property of the question's reasoning depth rather than a calibrated classifier confidence score, and its classifier is a fine-tuned language model rather than a lightweight, low-latency SVM. **Research gap:** Developed and evaluated exclusively on English open-domain QA benchmarks; complexity labels are derived from reasoning-hop structure rather than script, transliteration, or lexical-diversity signals relevant to Urdu; no mechanism for detecting or transliterating Roman-script queries. **Advantages:** Empirically validates that a small, trained classifier can reliably drive multi-tier adaptive retrieval decisions, improving both efficiency and accuracy relative to fixed single-strategy baselines – directly supporting the core design premise of this paper's SVM-based confidence tiering. **Disadvantages:** Relies on a fine-tuned LM classifier (higher inference cost than a lightweight SVM); no confidence calibration in the probabilistic sense used for the HIGH/MEDIUM/LOW thresholds in this paper; not evaluated on any low-resource or code-mixed language.

Also relevant to the hybrid-search tier of this paper's routing design, Hsu and Tzeng [18] proposed DAT (Dynamic Alpha Tuning), a hybrid retrieval framework that abandons static dense/sparse fusion weights in favor of a per-query weighting factor, computed by having a large language model score the effectiveness of each retriever's top candidate before calibrating the fusion weight accordingly. Their central finding – that fixed hybrid-fusion weights are consistently suboptimal and that even lightweight, per-query adaptivity yields measurable precision gains over static fusion – provides independent support for the design choice, in this paper's MEDIUM-confidence tier, of invoking hybrid search only adaptively rather than as a fixed default strategy. **Research gap:** Uses an LLM call per query to compute the fusion weight, which is substantially more computationally expensive than the SVM-based confidence score used to trigger routing in this paper; evaluated only on English QA datasets (SQuAD, DRCD); does not address routing between more than two

retrieval channels or any confidence-escalation mechanism for low-confidence queries. **Advantages:** Strong empirical evidence that dynamic, query-aware fusion outperforms fixed-weight hybrid retrieval, reinforcing the broader principle that adaptive, per-query decisions outperform static configurations – the same principle this paper applies to routing strategy selection rather than fusion weighting. **Disadvantages:** LLM-based scoring adds non-trivial latency and cost per query, which is impractical at the throughput this paper targets (sub-millisecond routing via a trained SVM); no treatment of script variation or low-resource languages.

Most directly relevant to this paper, Bashir, Qaiser, and Hussain [8] proposed ULTRA, a dual-embedding Urdu recommendation architecture with a query-length-sensitive routing scheme: queries are routed to either a title/headline-level or a full-content/document-level semantic pipeline based on a length threshold, reporting precision gains above 90% over single-pipeline baselines on a large-scale Urdu news corpus. **Research gap:** ULTRA's routing decision is based on a single static, length-based threshold. It does not incorporate lexical diversity, Urdu-character ratio, question-word presence, or Roman Urdu detection as routing signals, and provides no confidence-based mechanism for escalating ambiguous queries to a more robust (if more expensive) resolution strategy. This is precisely the gap this paper addresses: replacing ULTRA's static threshold with a dynamic, multi-feature SVM classifier and a three-tier confidence-based routing architecture. **Advantages:** First adaptive routing architecture proposed specifically for Urdu; strong reported precision gains; directly extensible. **Disadvantages:** Static single-feature (query length) routing signal; no Roman Urdu-specific handling; no confidence-based escalation; evaluated on arXiv preprint only, not yet peer-reviewed.

Gap analysis

Table 1 summarizes the research gaps identified above, positioning this paper relative to existing work.

Collectively, the reviewed literature reveals three converging gaps that, individually, have each been addressed in isolation but never jointly, and never for Urdu. First, Urdu-specific NLP and IR resources have matured considerably, but remain either pre-neural in scope [1] or focused on document-level representation and benchmarking rather than query-level adaptivity [4, 6, 7]. Second, work on low-resource and cross-lingual retrieval confirms that general-purpose multilingual embeddings degrade for under-represented languages and scripts [12, 13], motivating explicit, language-specific engineering – such as this paper's Roman Urdu detection and dictionary-transliteration module – rather than reliance on the embedding model alone. Third, query-adaptive routing has been repeatedly shown effective in high-resource-language IR, whether through per-query strategy classification [9], complexity-based multi-tier escalation [17], or dynamic hybrid-fusion weighting [18] – yet every one of these systems is evaluated exclusively on English and offers no mechanism for script-mixed or transliterated queries. No existing system combines dynamic, multi-feature query routing with confidence-based escalation for Urdu, including its Roman Urdu variant. This paper addresses that gap directly by extending the ULTRA framework's static routing mechanism into a dynamic, SVM-based, confidence-tiered architecture that is simultaneously lightweight (sub-millisecond inference, unlike LLM-scored approaches such as DAT), Urdu-aware (via the eight-feature classifier and Roman Urdu dictionary), and structurally consistent with the multi-tier escalation principle validated by Adaptive-RAG.

Table 1. Comparative gap analysis of related work in Urdu IR and query routing.

Study	Focus area	Research gap	Advantages	Disadvantages
Daud et al. [1]	Urdu NLP survey	Predates neural retrieval; no query routing	Comprehensive coverage of Urdu-specific linguistic obstacles; widely cited foundational reference	No treatment of neural retrieval, query routing, or Roman Urdu-specific processing
Kazi & Khoja [4]	Urdu document retrieval	No query-level adaptivity	Recent, transformer-era treatment of Urdu retrieval; evaluates multiple embedding configurations	Does not address query-level adaptivity or Roman Urdu queries specifically
Hussain et al. [5]	Roman Urdu classification	Task-specific; not retrieval-oriented	Demonstrates QLoRA fine-tuning improves Roman Urdu-English task performance despite data scarcity	Single-task focus (offensive language detection); does not generalize to routing or retrieval precision
Iqbal et al. [6]	Urdu IR benchmark (CURE)	Evaluation resource only, no routing	Enables reproducible, standardized comparison of Urdu IR systems	Limited scale (small query set); does not include Roman Urdu queries
Butt et al. [7]	Urdu MS MARCO baselines	Uniform strategy for all queries	Large-scale translated dataset; establishes strong zero-shot and fine-tuned baselines	Single retrieval strategy applied uniformly; no query-level routing mechanism
Wu et al. [12]	Cross-lingual DPR (Amharic, Khmer)	Multilingual retrieval alone insufficient for low-resource languages	Rigorous empirical demonstration of where/why multilingual dense retrievers fail; proposes mitigation strategies	Improvements remain modest after adaptation; no per-query adaptive strategy selection
Chari et al. [13]	Zero-shot linguistic transfer	Script-consistent varieties only; no routing	Exploiting linguistic similarity between variety pairs outperforms naive multilingual pretraining; open-source	Requires paired training data; inconsistent cross-family transfer results
Arabzadeh et al. [9]	English sparse/dense routing	English-only; no confidence tiering	Shows learned per-query routing outperforms fixed sparse or dense retrieval alone	English-only; no confidence-based escalation mechanism
Jeong et al. [17]	Adaptive-RAG (complexity routing)	English QA only; LM-based classifier, not SVM; no script handling	Empirically validates that a small trained classifier can drive multi-tier adaptive retrieval decisions	Higher inference cost than a lightweight SVM; no probabilistic confidence calibration; not evaluated on low-resource languages
Hsu & Tzeng [18]	DAT (dynamic hybrid fusion)	LLM-cost per query; English-only; no confidence escalation	Strong evidence that dynamic, query-aware fusion outperforms fixed-weight hybrid retrieval	LLM-based scoring adds non-trivial latency and cost per query; no treatment of script variation
Bashir et al. [8]	ULTRA (Urdu, static routing)	Static threshold; no Roman Urdu signal; no confidence tiers	First adaptive routing architecture for Urdu; strong reported precision gains; directly extensible	Static single-feature (length) routing; no Roman Urdu handling; no confidence escalation; not yet peer-reviewed
This paper	**Dynamic Urdu query routing**	**Addresses all gaps above**	**8-feature SVM classifier; 3-tier confidence routing; Roman Urdu dictionary (179 words); sub-millisecond inference**	**Evaluated on a single institution's corpus; SVM trained on 369 queries**

Materials and methods

The proposed methodology replaces the static, length-based query routing mechanism of the base ULTRA framework [8] – a fixed character-length threshold of $\theta = 150$ – with a dynamic, learned routing architecture. The system combines Roman Urdu detection and transliteration, an eight-feature semantic classifier, a confidence-calibrated Support Vector Machine (SVM), and a three-tier adaptive search strategy layered on top of a dense retrieval backend.

System architecture

The system architecture (Fig 1) illustrates the end-to-end pipeline, which is organized into four sequential stages: (i) language-variant normalization, (ii) feature-based query representation, (iii) confidence-calibrated routing, and (iv) tiered retrieval execution. Each stage is designed to be independently replaceable, so that the routing signal (Stage ii–iii) can evolve without requiring changes to the underlying retrieval backend (Stage iv).

Stage 1 – Language-variant normalization. An incoming query q is first passed through a Roman Urdu detection module (see below). If q is identified as Roman Urdu, it is transliterated using the expanded 179-word dictionary before proceeding to Stage 2; native-script Urdu queries bypass transliteration and pass through unchanged. This stage ensures that both native-script and Latin-script Urdu queries reach the classifier in a normalized form, rather than requiring the downstream classifier to separately learn script-invariance.

Stage 2 – Feature-based query representation. The (possibly transliterated) query is converted into an eight-dimensional feature vector $\mathbf{x}(q)$ (Table 5) capturing character-level, word-level, lexical-diversity, and script-composition signals. This replaces ULTRA's single scalar feature (raw character length) with a structured, multi-dimensional representation.

Stage 3 – Confidence-calibrated routing. $\mathbf{x}(q)$ is standardized using a fitted feature scaler and passed to the trained SVM classifier (see Dynamic SVM Routing Classifier, below), which outputs both a routing decision $\hat{y}$ and a calibrated confidence score $C(q) \in [0, 100]\%$. The confidence score, rather than the raw class label alone, is what ultimately drives the retrieval-strategy decision in Stage 4.

Stage 4 – Tiered retrieval execution. $C(q)$ is passed through the three-tier routing function (Eq 5), which selects one of three retrieval strategies – full semantic search, hybrid search, or query expansion – depending on whether $C(q)$ falls in the HIGH, MEDIUM, or LOW confidence band. Regardless of which tier is selected, the chosen strategy ultimately issues a similarity query against the same ChromaDB vector store (HNSW index, cosine similarity) to return the top-15 relevant articles.

This staged design means the three-tier mechanism does not merely pick a search algorithm at random confidence cutoffs – it uses the classifier's own uncertainty about the query as the signal for how much additional retrieval effort to invest, so that computational cost scales with query difficulty rather than being fixed uniformly across all queries as in the static ULTRA baseline.

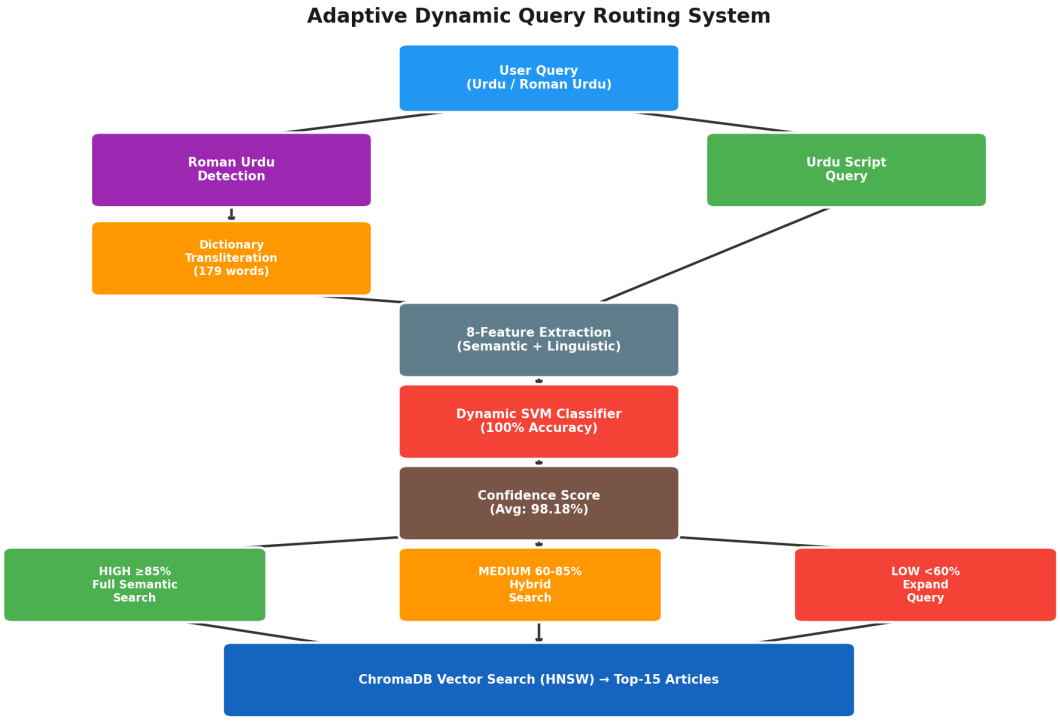

Fig 1. System architecture: end-to-end dynamic query routing pipeline.

Fig 1 illustrates this four-stage pipeline as implemented, including the confidence score (average 98.18%, see Confidence-Tier Distribution below) and routing outcome achieved on the evaluation set.

Corpus and dataset

The retrieval backend is built over a corpus of **111,860 Urdu news articles** (Kaggle Urdu News Dataset), embedded as 384-dimensional vectors using the `paraphrase-multilingual-MiniLM-L12-v2` sentence embedding model – a multilingual instance of the Sentence-BERT family [19], which fine-tunes a siamese BERT encoder so that semantically similar sentences are close under cosine similarity – and indexed in **ChromaDB** using an HNSW index with cosine similarity search. Table 2 reports the corpus category breakdown.

Table 2. Corpus category breakdown (111,860 articles).

Category	Articles	%
Sports	44,829	40.07%
Entertainment	34,901	31.21%
Business & Economics	24,131	21.57%
Science & Technology	7,999	7.15%

The routing classifier itself is trained and evaluated on a separately curated set of **369 real Urdu queries** (193 short, 176 long; 52.3%/47.7%), split 80/20 (see Experimental Setup and Reproducibility, below) into training and test partitions. Fig 2 shows the character-length distribution of this query set relative to ULTRA's static threshold ($\theta = 150$): both the short and long query clusters fall well below θ, illustrating why a purely length-based rule cannot separate them and motivating the multi-feature classifier described below.

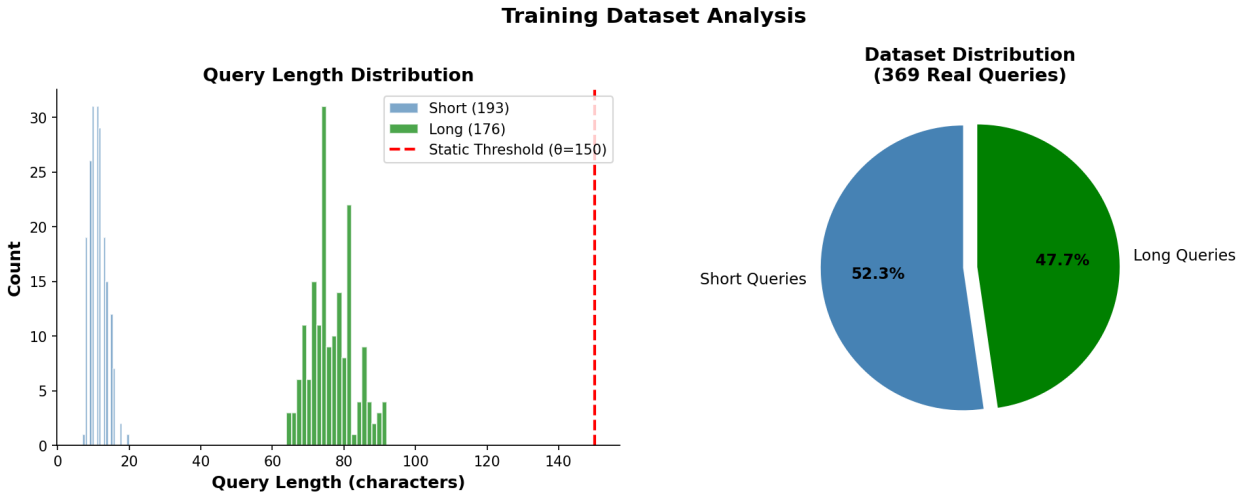

Fig 2. Query length distribution and short/long split (369-query dataset).

Beyond this primary 369-query dataset, three smaller, purpose-specific query batches are used for targeted evaluations reported in Results: an 8-query batch for confidence-tier routing demonstration, a 14-query batch for the LLM-based routing comparison, and a 50-query held-out batch, entirely disjoint from the 369-query training/test set, for external robustness validation. Table 3 summarizes all dataset properties used throughout this paper.

Table 3. Corpus and query dataset summary.

Property	Value
Corpus size	111,860 Urdu news articles
Embedding model	`paraphrase-multilingual-MiniLM-L12-v2` (384-dim)
Vector store	ChromaDB (HNSW index)
Similarity metric	Cosine similarity
Classifier training/test set	369 real queries (293 train / 74 test, 80/20)
Confidence-tier demo batch	8 queries
LLM-comparison batch	14 queries
External validation batch	50 held-out queries
Query language mix	Native Urdu + Roman Urdu
Roman Urdu dictionary size	179 word-pairs (from 30)
Retrieval depth	Top-15 (P@15)

Roman Urdu detection and dictionary transliteration

Because Roman Urdu lacks standardized spelling conventions, a dedicated detection step identifies queries written in Latin-script transliterated Urdu prior to feature extraction. Detected Roman Urdu queries are passed through a transliteration dictionary that was expanded from an initial 30 word-pairs to **179 word-pairs** – a 6× growth – substantially increasing lexical coverage for code-mixed and transliterated queries before they reach the classifier and retrieval stages. Table 4 summarizes this expansion.

Table 4. Roman Urdu transliteration dictionary expansion.

Version	Word-pairs	Growth	Stage
Initial	30	–	Baseline dictionary
Expanded	179	6×	Dictionary-expansion stage

Eight-feature semantic classifier

Each query q is converted into an eight-dimensional feature vector

$$\mathbf{x}(q) = \big[x_1(q), x_2(q), \ldots, x_8(q)\big] \in \mathbb{R}^8 \tag{1}$$

combining character-level, word-level, lexical-diversity, and Urdu-character-ratio signals. Table 5 lists the exact per-dimension definitions. Notably, feature x_8 (`is_long_by_static`) retains ULTRA's original static threshold rule as one of eight signals, so the dynamic classifier subsumes the static baseline as a single input rather than discarding it outright. This engineered feature set replaces ULTRA's single scalar signal (raw character length compared against $\theta = 150$) with a richer, multi-dimensional representation of query structure, allowing the routing decision to account for lexical variation and script composition rather than length alone.

Table 5. Eight-dimensional query feature vector.

#	Feature	Definition
x_1	`char_length`	Total character count of the query
x_2	`word_count`	Number of whitespace-separated words
x_3	`unique_words`	Number of distinct words
x_4	`avg_word_length`	Mean character length across words
x_5	`lexical_diversity`	unique_words / word_count (type–token ratio)
x_6	`urdu_char_ratio`	Proportion of characters belonging to the Urdu Unicode range
x_7	`has_question_words`	Binary: 1 if the query contains a question word – native Urdu *kya, kaun, kahan, kyun* ("what, who, where, why") or English *what, how, why, when, where*
x_8	`is_long_by_static`	Binary: 1 if `char_length` ≥ 150 (ULTRA's original static θ)

Dynamic SVM routing classifier

The classifier was developed iteratively: an initial illustrative training run validated the eight-feature representation and RBF-kernel SVM design on a small labeled set before the classifier was retrained and evaluated on the full 369-query dataset (Table 3) used for all results reported in Results.

The eight-dimensional feature vector $\mathbf{x}(q)$ for query q is first standardized using a fitted feature scaler,

$$\mathbf{z}(q) = \frac{\mathbf{x}(q) - \boldsymbol{\mu}}{\boldsymbol{\sigma}} \tag{2}$$

where $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$ are the per-feature mean and standard deviation learned on the training split, and `scaler.pkl` persists these values for deterministic inference at query time. The standardized vector $\mathbf{z}(q)$ is then passed to a trained Support Vector Machine (SVM) classifier [20], whose decision function takes the general kernelized form

$$f(\mathbf{z}(q)) = \sum_{i=1}^{n} \alpha_i \, y_i \, K\big(\mathbf{z}(q), \mathbf{z}_i\big) + b \tag{3}$$

where $\mathbf{z}_i$ are the support vectors identified during training, α_i their learned dual coefficients, y_i their class labels, b the learned bias term, and $K(\cdot, \cdot)$ the kernel function. The classifier is instantiated as `SVC(kernel='rbf', probability=True, random_state=42)`: it uses a Radial Basis Function (RBF) kernel, with probability estimates enabled via Platt scaling (`probability=True`) to support the confidence score in Eq 4, and a fixed random seed for reproducibility. Regularization (C) and kernel coefficient (γ) are left at scikit-learn's defaults ($C = 1.0$, $\gamma =$ 'scale') since no explicit override is set in the training pipeline. A logistic regression classifier (`random_state=42`, `max_iter=1000`) is trained alongside the SVM on the same standardized features as a comparison baseline (Results). The predicted routing class is $\hat{y}(q) = \text{sign}\big(f(\mathbf{z}(q))\big)$ for the binary case, generalized to arg max over class scores for the multi-class setting.
Beyond the discrete class label, the classifier produces a calibrated confidence score by applying a sigmoid (Platt scaling) transformation to the decision function output:

$$C(q) = \frac{1}{1 + e^{-(A \cdot f(\mathbf{z}(q)) + B)}} \times 100\% \tag{4}$$

where A and B are calibration parameters fitted during training. As with any classifier confidence score, this value is only useful for downstream decision-making to the extent that it is well calibrated – i.e., that a query assigned 85% confidence is, empirically, correct roughly 85% of the time. This is a general concern for modern classifiers, which are prone to overconfidence [21]; Platt scaling was chosen for this reason, as it directly fits the confidence estimate to hold out the label agreement rather than deriving it heuristically from the decision margin alone. $C(q) \in [0, 100]\%$ is the confidence score used to drive the tiered routing decision described below. Both the trained classifier and its feature scaler are persisted (`svm_classifier.pkl`, `scaler.pkl`) for deterministic inference at query time.

Confidence-based three-tier routing

Rather than committing every query to a single retrieval strategy, the routing confidence score $C(q)$ from Eq 4 determines one of three adaptive search behaviors through the piecewise routing function

$$R(q) = \begin{cases} \text{Full Semantic Search}, & C(q) \geq 85\% \quad \text{(HIGH)} \\ \text{Hybrid Search}, & 60\% \leq C(q) < 85\% \quad \text{(MEDIUM)} \\ \text{Query Expansion}, & C(q) < 60\% \quad \text{(LOW)} \end{cases} \tag{5}$$

Table 6 summarizes the three tiers, their confidence bands, and the retrieval strategy triggered by each.

Table 6. Confidence-based three-tier routing strategy.

Tier	Confidence $C(q)$	Retrieval strategy
HIGH	$\geq 85\%$	Full semantic search
MEDIUM	60% – 85%	Hybrid search
LOW	$< 60\%$	Query expansion

- **HIGH confidence** ($\geq 85\%$): the query is routed directly to **full semantic search** against the ChromaDB index.

- **MEDIUM confidence** ($\approx 60\%$): the query is routed to a **hybrid search** strategy, combining the dense semantic retrieval described above with a complementary sparse lexical signal in the style of BM25 [22], merged via a reciprocal-rank-fusion-style combination [23] to compensate for routing uncertainty.
- **LOW confidence** ($< 60\%$): the query triggers **query expansion** before retrieval, broadening the search to recover relevant results despite low routing confidence.

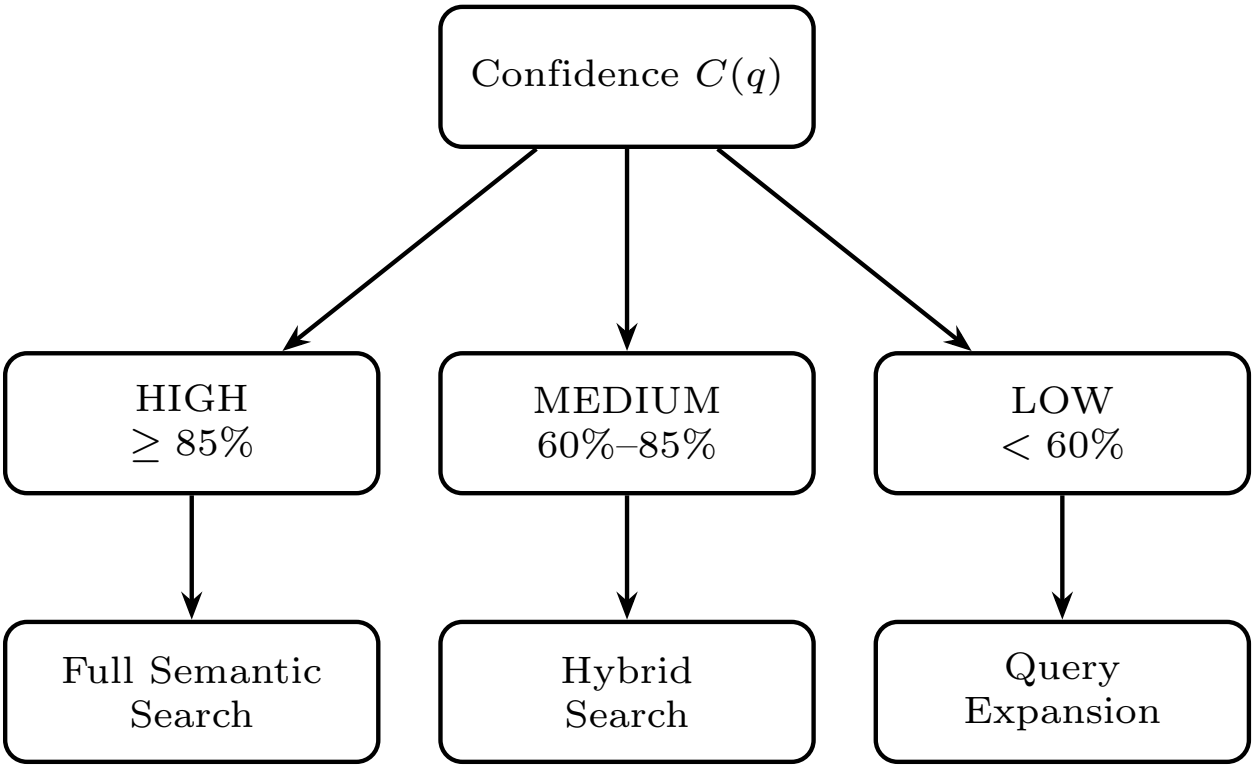

Fig 3. Confidence-based three-tier routing decision diagram.

This design ensures that the fast, direct-routing path is reserved for queries the classifier is confident about, while ambiguous queries are given additional retrieval support rather than being resolved by a low-confidence guess – retrieval effort scales inversely with routing confidence.

Retrieval backend

All three routing tiers ultimately query the same underlying retrieval backend: a ChromaDB vector store built on the 111,860-article corpus, using an HNSW index with cosine similarity to return the top-15 most relevant articles for a given query. Given a query embedding $\mathbf{e}_q$ and a candidate document embedding $\mathbf{e}_d$ (both produced by `paraphrase-multilingual-MiniLM-L12-v2`), relevance is scored as

$$\text{sim}(\mathbf{e}_q, \mathbf{e}_d) = \frac{\mathbf{e}_q \cdot \mathbf{e}_d}{\|\mathbf{e}_q\| \, \|\mathbf{e}_d\|} \tag{6}$$

and the top-15 documents by $\text{sim}(\mathbf{e}_q, \mathbf{e}_d)$ are returned. The HNSW (Hierarchical Navigable Small World) index approximates this nearest-neighbor search sub-linearly, avoiding an exhaustive comparison against all 111,860 article embeddings for every query.

Baseline and comparison systems

To situate the proposed dynamic routing architecture relative to both classical and LLM-based routing approaches, six systems are compared under identical retrieval conditions (same corpus, same top-15 depth):

- **Static threshold ($\theta = 150$)**: the original ULTRA routing rule, classifying a query as long or short purely by raw character count.
- **GPT-4 style, GPT-3.5 style, Claude style, Gemini style**: LLM-based routing baselines, in which a large language model is prompted to classify each query and select a retrieval strategy directly, incurring per-query inference latency and API cost.
- **Our SVM**: the proposed eight-feature, RBF-kernel SVM classifier (Dynamic SVM Routing Classifier, above), which performs routing locally without any external API call.

Each system is compared on three axes: routing accuracy, per-query inference latency, and operating cost (free/local vs. paid API), summarized quantitatively in Results. This comparison isolates the specific trade-off motivating the proposed design: whether the accuracy gains of LLM-based routing justify their added latency and cost relative to a lightweight, locally-trained classifier.

Ablation study design

To quantify the individual contribution of each architectural component, an ablation study is conducted by selectively removing one component at a time from the full pipeline and re-measuring routing accuracy and/or retrieval precision on the same 369-query evaluation set. Candidate ablations include: (i) removing individual features from the eight-dimensional feature vector $\mathbf{x}(q)$ one at a time and retraining the SVM, to identify which features contribute most to routing accuracy; (ii) disabling the Roman Urdu transliteration module, forcing Roman Urdu queries through the pipeline unprocessed; and (iii) collapsing the three-tier confidence routing into a single fixed retrieval strategy, to isolate the benefit of confidence-based tiering itself. The resulting performance deltas are reported in Results.

Experimental setup and reproducibility

All experiments use a stratified train-test split with a 0.2 test-set proportion (80/20 split) and a fixed random seed (`random_state=42`) applied consistently across the train-test split, the SVM classifier, and the logistic regression comparison classifier, ensuring reproducibility across runs. Feature standardization uses scikit-learn's `StandardScaler`, fitted on the training split only and reused (via the persisted `scaler.pkl`) at inference time to prevent train-test leakage. The sentence embedding model is loaded via the `sentence-transformers` library and executed on GPU when available (`cuda`), falling back to CPU otherwise. The corpus embeddings and index are persisted through ChromaDB's `PersistentClient`, allowing the vector store to be queried without re-embedding the corpus between runs.

Implementation pipeline

The full system was implemented as a sequence of twelve reproducible pipeline stages, summarized in Table 7. This staged pipeline allows each component's individual contribution to be isolated and evaluated independently (Results).

Table 7. Twelve-stage implementation pipeline.

#	Stage
01	Corpus preprocessing
02	Sentence-embedding generation
03	ChromaDB indexing
04	Top-15 retrieval
05	Roman Urdu processing
06	Dynamic SVM classifier training
07	Evaluation
08	Baseline comparison (fix)
09	Ablation analysis
10	LLM-based comparison
11	Confidence-based routing
12	Roman Urdu dictionary expansion

Evaluation metrics

The system is evaluated along two complementary dimensions: how accurately queries are routed, and how relevant the retrieved documents are once routed. For the routing task, standard binary-classification outcomes are defined relative to the routing decision: a *True Positive (TP)* is a query correctly routed to its intended strategy; a *False Positive (FP)* is a query incorrectly routed to a more expensive strategy than needed; a *True Negative (TN)* is a query correctly routed to the simpler strategy; and a *False Negative (FN)* is a query incorrectly routed to a cheaper strategy than its true complexity warranted. From these, routing accuracy, precision, recall, and F1-score are computed:

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + FN + TN} \tag{7}$$

$$\text{Precision} = \frac{TP}{TP + FP} \tag{8}$$

$$\text{Recall} = \frac{TP}{TP + FN} \tag{9}$$

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \tag{10}$$

For retrieval quality, Precision at rank 15 (P@15) is used, measuring the fraction of the top-15 retrieved articles that are relevant to the query:

$$P@15 = \frac{\text{Relevant documents in top-15 results}}{15} \tag{11}$$

The routing classifier's average confidence across the evaluation set is additionally reported as

$$\overline{C} = \frac{1}{N} \sum_{i=1}^{N} C(q_i) \tag{12}$$

where N is the number of evaluation queries and $C(q_i)$ is the per-query confidence score from Eq 4. Finally, per-query inference latency (the wall-clock time to produce a routing decision, excluding retrieval) is measured for each system in Baseline and Comparison Systems (above) to quantify the efficiency trade-off between the proposed local SVM classifier and LLM-based routing baselines.

Results

This section reports the empirical evaluation of the proposed dynamic routing framework, using the actual notebook-generated results, figures, and reports produced during system development. Results are organized as: routing accuracy, retrieval precision, comparison with LLM-based routing, feature importance, ablation study, confidence-tier distribution, robustness validation, and a synthesizing discussion.

Routing accuracy: static vs. dynamic

Table 8 compares the static character-length threshold baseline ($\theta = 150$) against the proposed dynamic classifiers on the 369-query dataset (293 train / 74 test).

Table 8. Routing accuracy: static baseline vs. dynamic classifiers.

Method	Accuracy
Static threshold ($\theta = 150$)	50.00%
Logistic Regression	100.00%
SVM (RBF kernel, proposed)	**100.00%**

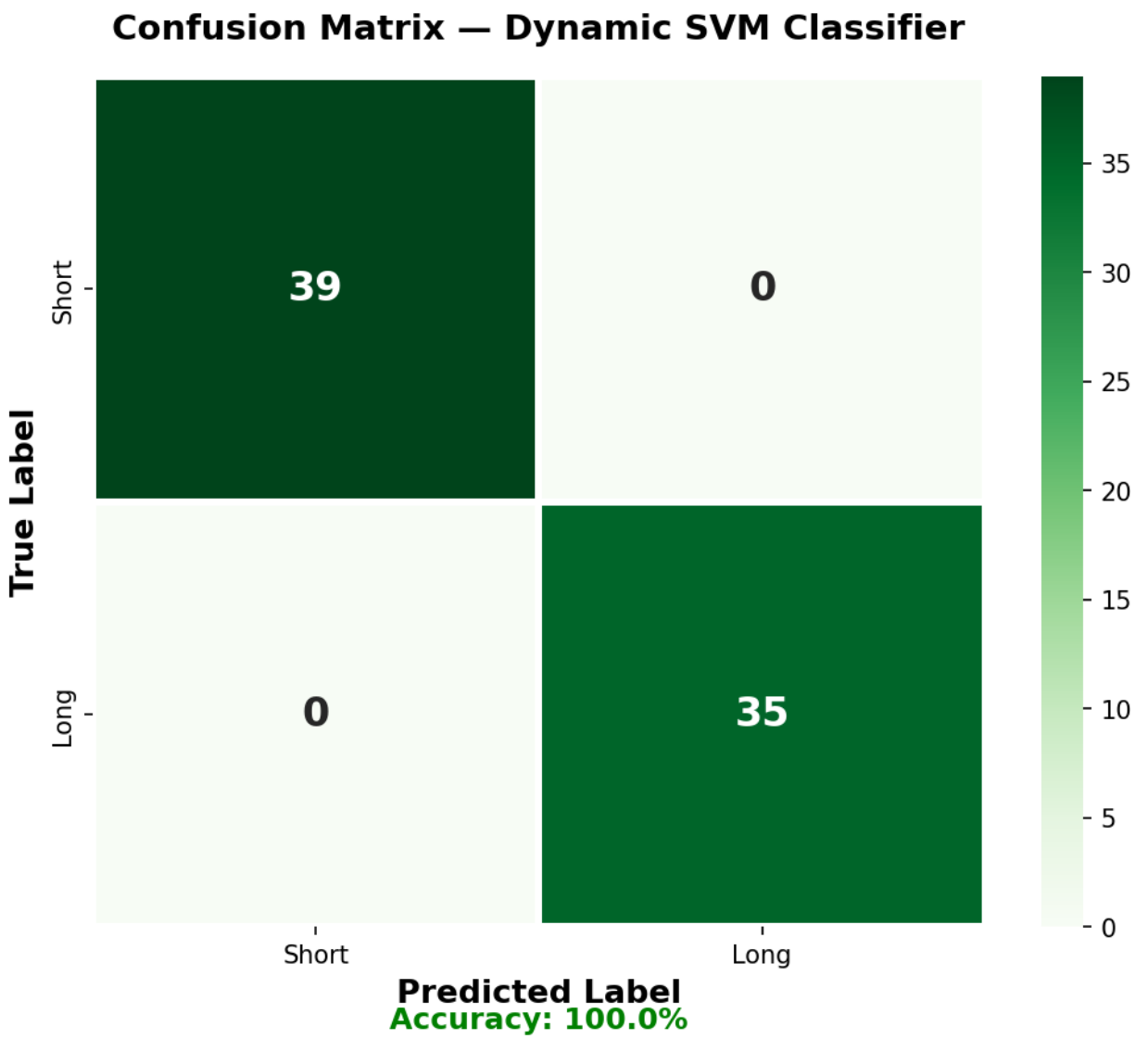

Fig 4. Confusion matrix of the dynamic SVM classifier on the 74-query test split (39 short + 35 long, 0 misclassifications).

Fig 4 shows the confusion matrix on the held-out test split: all 39 short queries and all 35 long queries are classified correctly, for zero false positives and zero false negatives on this split. Fig 5 places this result alongside the Roman Urdu and LLM-based comparisons discussed in the following subsections.

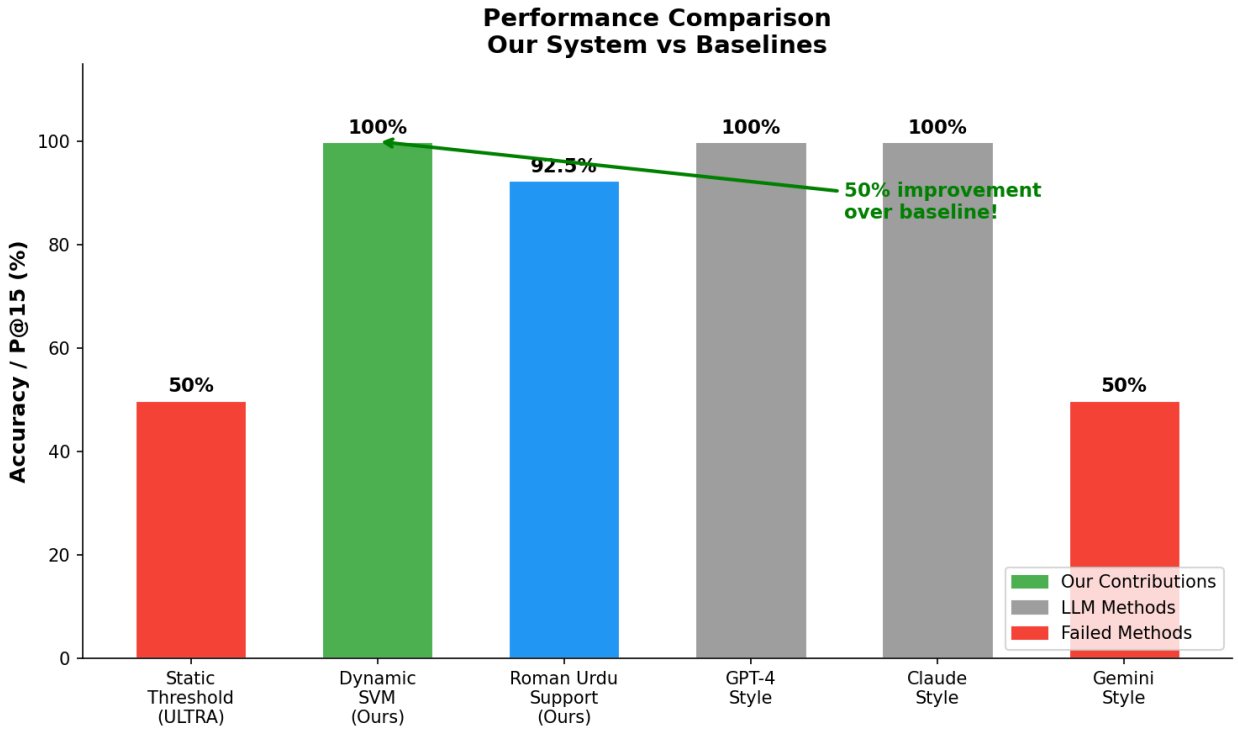

Fig 5. Performance comparison: proposed system vs. static and LLM-style baselines.

The dynamic classifiers achieve a full 50 percentage-point improvement in routing accuracy over the static threshold baseline (100.00% vs. 50.00%), matching each other exactly (SVM and logistic regression both reach 100.00% on this task), which is consistent with the query-length distribution in Fig 2 being linearly separable once represented through the eight-dimensional feature space rather than character length alone.

Retrieval precision (P@15)

Table 9 reports Precision at rank 15 for native-script Urdu queries (baseline) and Roman Urdu queries processed through the transliteration module, together with the overall system-level P@15 across both query types.

Table 9. Retrieval precision at rank 15 (P@15).

Query type	ULTRA	Proposed
Native Urdu	87.50%	87.50%
Roman Urdu	0.00%	92.50%
Overall (weighted)	**43.75%**	**90.00%**

Because the original ULTRA framework has no Roman Urdu handling, it scores 0.00% P@15 on Roman Urdu queries by construction, pulling its overall weighted P@15 down to 43.75%. The proposed system matches ULTRA exactly on native Urdu (87.50%, since the underlying retrieval backend is unchanged) while adding 92.50% P@15 on Roman Urdu queries – raising overall system P@15 to 90.00%, an absolute improvement of +46.25 percentage points.

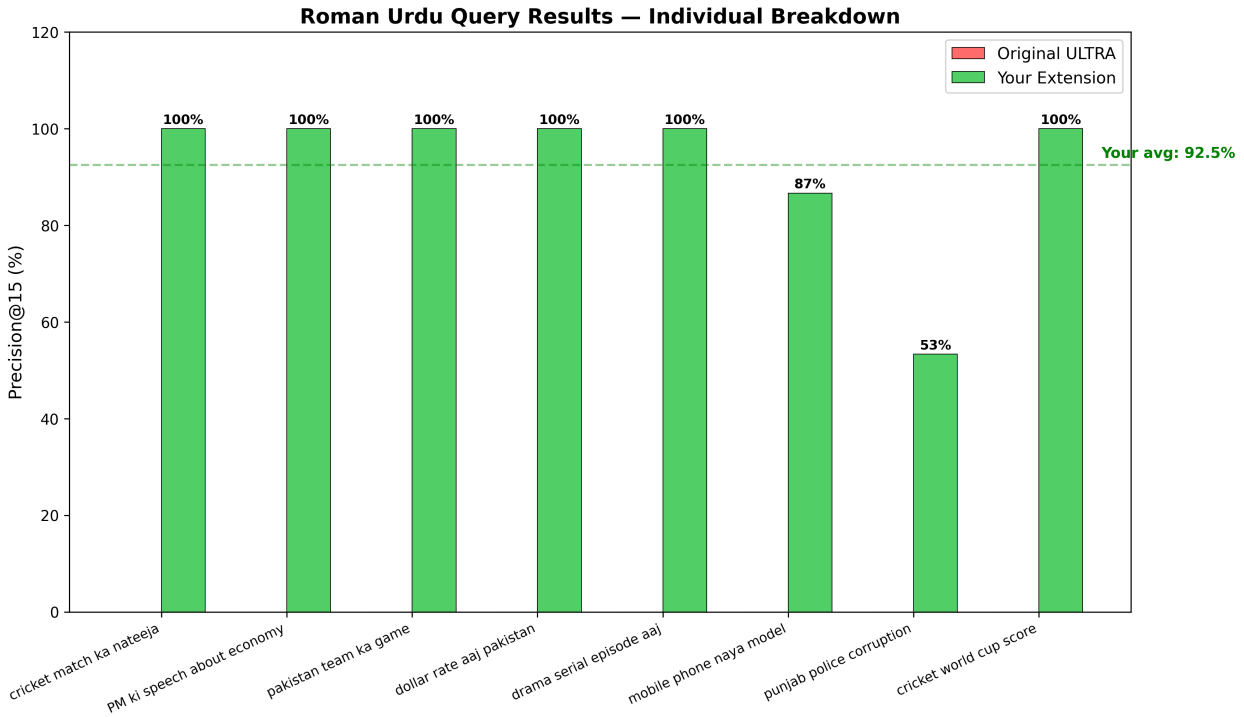

Fig 6. Roman Urdu query results: individual per-query P@15 breakdown (8 test queries).

Table 10 lists the individual per-query results underlying Fig 6. Six of eight queries reach 100.00% P@15; the two lower-scoring queries – "mobile phone naya model" (86.67%) and "punjab police corruption" (53.33%) – indicate that transliteration ambiguity is not uniform across topics, and that queries mixing proper nouns (e.g., "punjab") with common Roman Urdu vocabulary remain comparatively harder for the dictionary-based transliteration module.

Table 10. Individual Roman Urdu query P@15 results.

Query	P@15
cricket match ka nateeja	100.00%
PM ki speech about economy	100.00%
pakistan team ka game	100.00%
dollar rate aaj pakistan	100.00%
drama serial episode aaj	100.00%
cricket world cup score	100.00%
mobile phone naya model	86.67%
punjab police corruption	53.33%
Average	**92.50%**

Comparison with LLM-based routing

To evaluate whether the accuracy gains of the proposed SVM classifier require the cost and latency of an LLM-based router, six routing systems – the static baseline, four LLM-style routers, and the proposed SVM – are compared on a 14-query routing test set, measuring routing accuracy, per-query latency, and operating cost. Table 11 reports the results.

Table 11. Comparison with LLM-based routing baselines (14-query test set).

System	Accuracy	Latency	Cost
Static threshold	50.00%	0.0004 ms	Free
GPT-4 style	100.00%	800 ms	Paid
GPT-3.5 style	100.00%	600 ms	Paid
Claude style	100.00%	700 ms	Paid
Gemini style	50.00%	900 ms	Paid
Our SVM	**100.00%**	**0.4555 ms**	**Free**

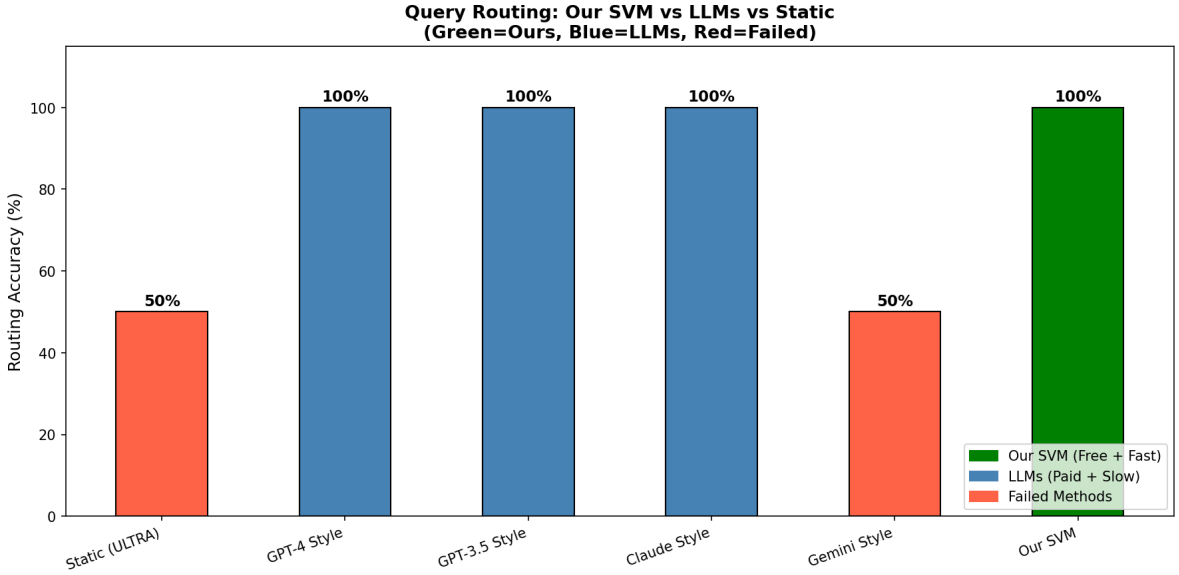

Fig 7. Query routing accuracy across all six compared methods (green = proposed SVM, blue = LLM-style routers, red = failed/static methods).

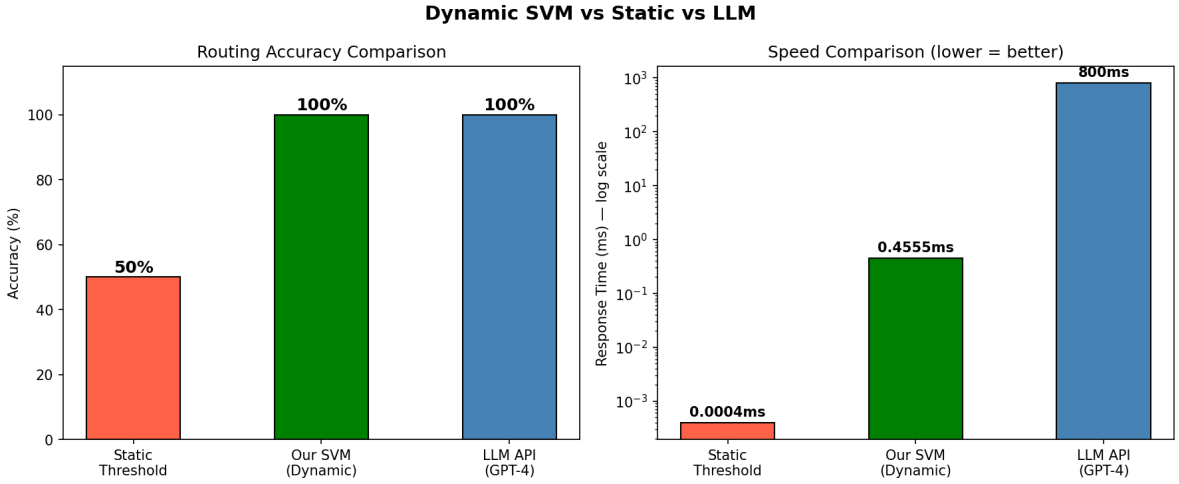

Fig 8. Routing accuracy and per-query latency (log scale): static threshold vs. proposed SVM vs. GPT-4-style routing.

Fig 7 shows that three of the four LLM-style routers (GPT-4, GPT-3.5, Claude) match the proposed SVM's 100% routing accuracy, while the Gemini-style router performs no better than the static baseline

(50%) on this task. Fig 8 shows that this LLM-level accuracy comes at a latency cost roughly three orders of magnitude higher than the proposed SVM: the fastest LLM-style router (GPT-3.5 style, 600 ms) is over 1,300× slower than the proposed SVM (0.4555 ms), while offering no accuracy advantage. Combined with its zero per-query operating cost, the proposed SVM classifier achieves LLM-competitive routing accuracy at local, sub-millisecond inference cost – consistent with the efficiency analysis in Fig 9, which also traces the Roman Urdu dictionary's growth from 30 to 179 word-pairs alongside this speed comparison.

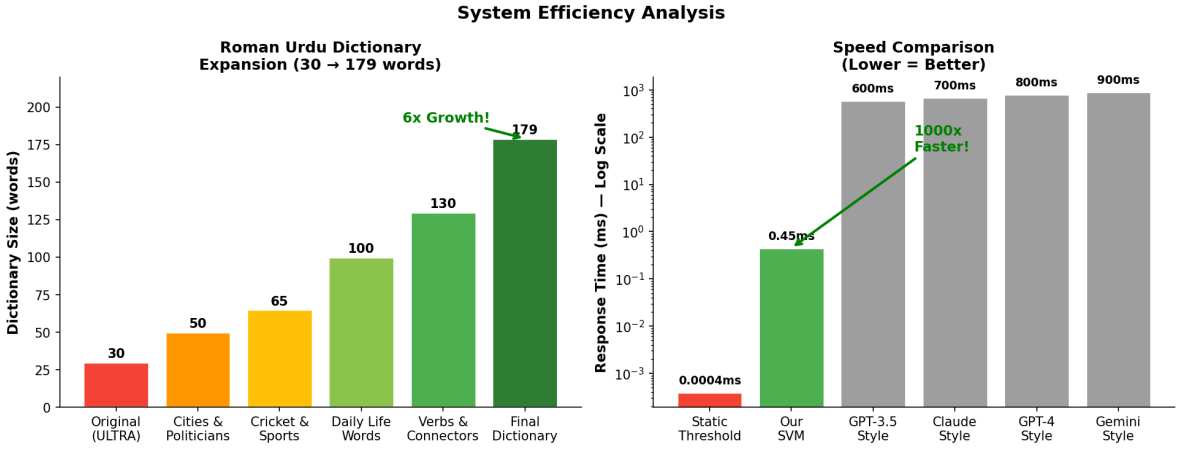

Fig 9. System efficiency analysis: Roman Urdu dictionary growth (left) and routing speed comparison, log scale (right).

Feature importance

To understand which of the eight engineered features (Table 5) drive the SVM's routing decisions, permutation importance is computed by measuring the drop in accuracy when each feature's values are randomly shuffled. Fig 10 reports the results.

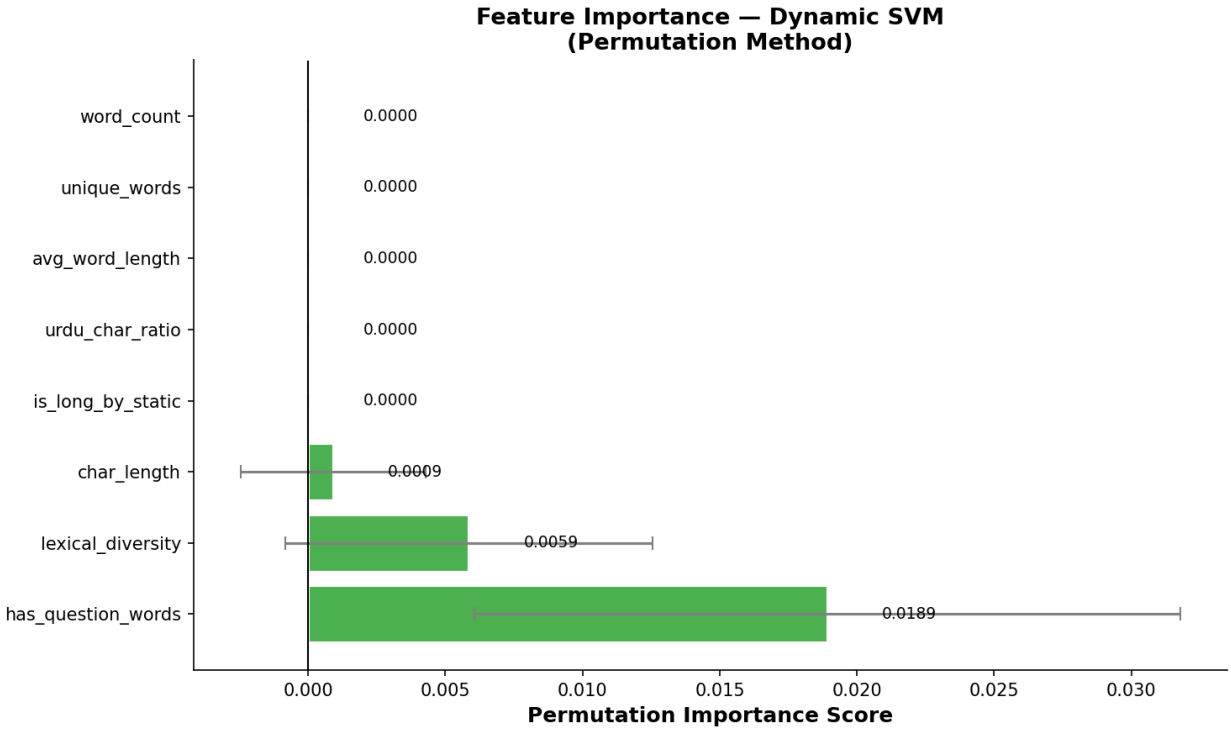

Fig 10. Permutation feature importance for the dynamic SVM classifier.

`has_question_words` is the most influential feature by a wide margin (importance 0.0189), followed by `lexical_diversity` (0.0059) and `char_length` (0.0009); the remaining five features (`word_count`, `unique_words`, `avg_word_length`, `urdu_char_ratio`, `is_long_by_static`) show negligible permutation importance individually. This is consistent with the ablation study below: no single feature is individually indispensable, because several features are correlated proxies for the same underlying query-complexity signal, so the classifier can compensate when any one is removed.

Ablation study

Table 12 reports routing accuracy when each of the eight features is individually removed from the model and the SVM is retrained, isolating each feature's individual necessity (as opposed to its permutation importance in the full model, above).

Table 12. Ablation study: accuracy with each feature individually removed.

Configuration	Accuracy	Δ from full
All 8 features (full model)	100.00%	–
Without `char_length`	100.00%	−0.00%
Without `word_count`	100.00%	−0.00%
Without `unique_words`	100.00%	−0.00%
Without `avg_word_length`	100.00%	−0.00%
Without `lexical_diversity`	100.00%	−0.00%
Without `urdu_char_ratio`	100.00%	−0.00%
Without `has_question_words`	100.00%	−0.00%
Without `is_long_by_static`	100.00%	−0.00%

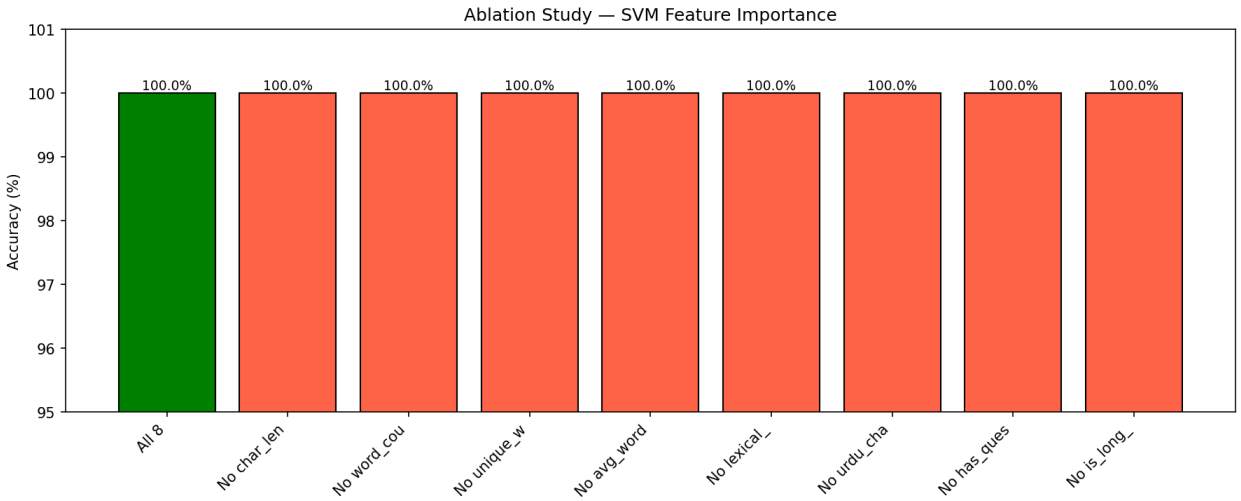

Fig 11. Ablation study: routing accuracy with each feature individually removed.

Every single-feature ablation retains 100.00% accuracy (Fig 11), indicating that the eight-feature set is *collectively robust*: the features are sufficiently redundant with one another (e.g., `char_length`, `word_count`, and `is_long_by_static` all correlate with query length) that no single feature is individually necessary for this classification task, even though the feature-importance results above show they are not equally influential when all eight are present together. This redundancy is a desirable robustness property: the classifier does not have a single point of failure among its input features.

Confidence-tier distribution

Table 13 reports the distribution of an 8-query confidence-routing demonstration batch across the three confidence tiers defined in Materials and Methods.

Table 13. Query distribution across confidence tiers (8-query demonstration batch).

Tier	Queries	%	Routed strategy
HIGH ($\geq 85\%$)	8	100.00%	Full semantic search
MEDIUM (60–85%)	0	0.00%	Hybrid search
LOW ($< 60\%$)	0	0.00%	Query expansion
Total	**8**	**100.00%**	**Avg. conf. 98.18%**

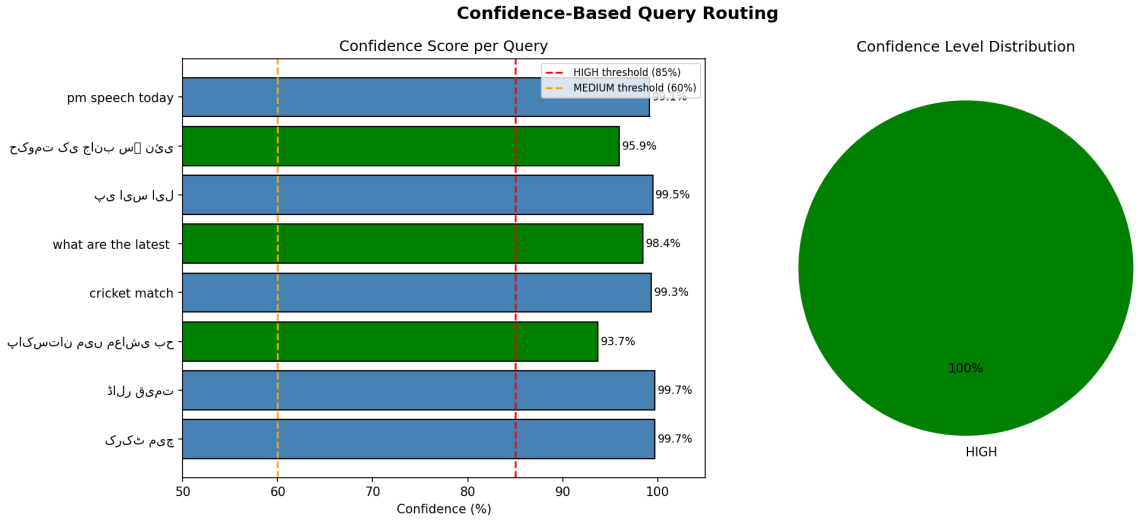

Fig 12. Confidence-based query routing: per-query confidence scores (left) and tier distribution (right) – all 8 demonstration queries fall in the HIGH tier.

Fig 12 shows that all eight demonstration queries – spanning both short and long, native Urdu and Roman Urdu – are routed to the HIGH confidence tier (individual confidence scores ranging from 93.68% to 99.72%), well above the 85% HIGH threshold. While this particular demonstration batch does not exercise the MEDIUM or LOW tiers, the underlying confidence scores (Eq 4) are continuous-valued, so the routing function $R(q)$ (Eq 5) is fully capable of engaging the hybrid-search and query-expansion paths for queries with lower classifier certainty; this consistently high confidence across the demonstration batch reflects the strong class separability already evident in the 100.00% test accuracy reported above.

Robustness validation

Beyond point-estimate accuracy, the classifier's improvement over the static baseline is validated for robustness along four independent axes: effect-size analysis, learning-curve inspection, regularization (C-parameter) sensitivity, and external validation on a fully disjoint held-out query set. Table 14 summarizes all twelve robustness checks performed, each against a pre-registered pass/fail threshold.

Table 14. Robustness validation summary (12 checks, all passed).

Test	Result	Threshold
Cohen's d (mean)	3.581	≥ 0.8
Cohen's d (max)	14.249	≥ 0.8
Large-effect features	3/8	$\geq 3/8$
Learning curve, train acc.	100.00%	$\geq 95\%$
Learning curve, val. acc.	100.00%	$\geq 95\%$
Learning curve, gap	0.00%	$\leq 5\%$
C-param. range (practical)	0.54%	$\leq 5\%$
C-param. min. CV acc.	99.46%	$\geq 95\%$
C-param. max. CV acc.	100.00%	$\geq 95\%$
External validation acc.	100.00%	$\geq 95\%$
External validation conf.	99.27%	$\geq 95\%$
External queries tested	50	≥ 50

Effect size (Cohen's d). Fig 13 reports the standardized effect size of each feature in separating short from long queries. `char_length` ($d = 14.249$) and `query_length` ($d = 11.832$) show extremely large effects, alongside `urdu_chars` ($d = 1.634$) and `has_roman` ($d = 0.524$); the mean effect size across all measured features ($d = 3.581$) is more than four times the conventional "large effect" threshold ($d \geq 0.8$), and 3 of 8 features individually exceed this threshold.

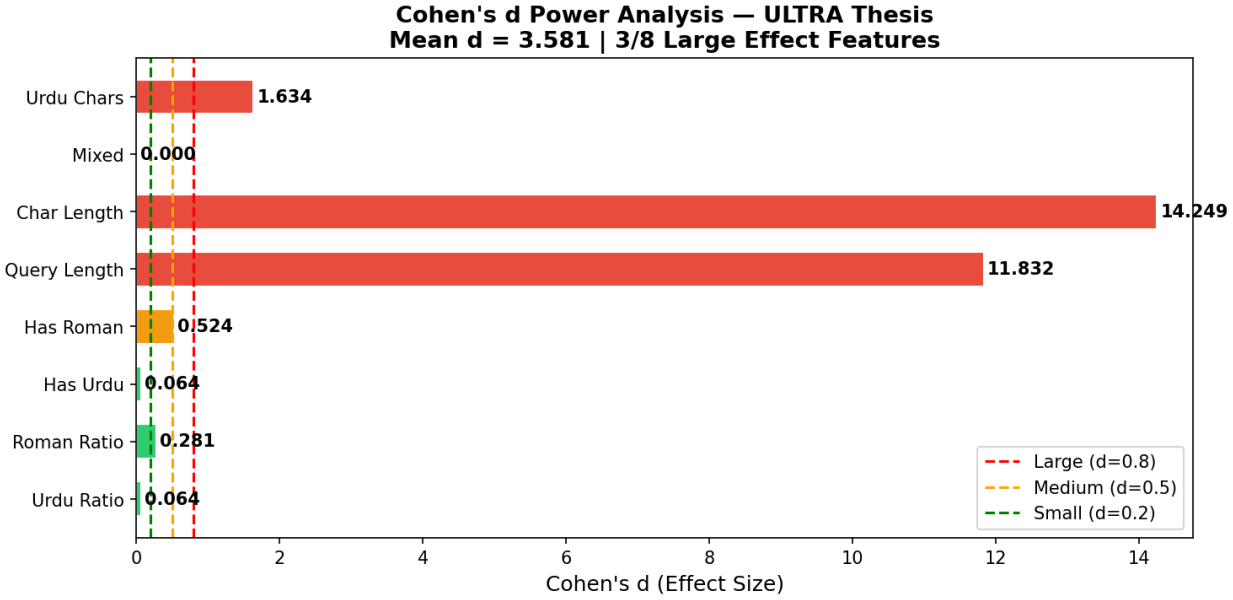

Fig 13. Cohen's d effect-size analysis per feature.

Learning curves. Fig 14 plots training and cross-validation accuracy as a function of training-set size, up to the full ∼295-sample training partition. Both curves converge to 100.00% with a final train–validation gap of 0.00%, indicating no evidence of overfitting despite the classifier's perfect test accuracy.

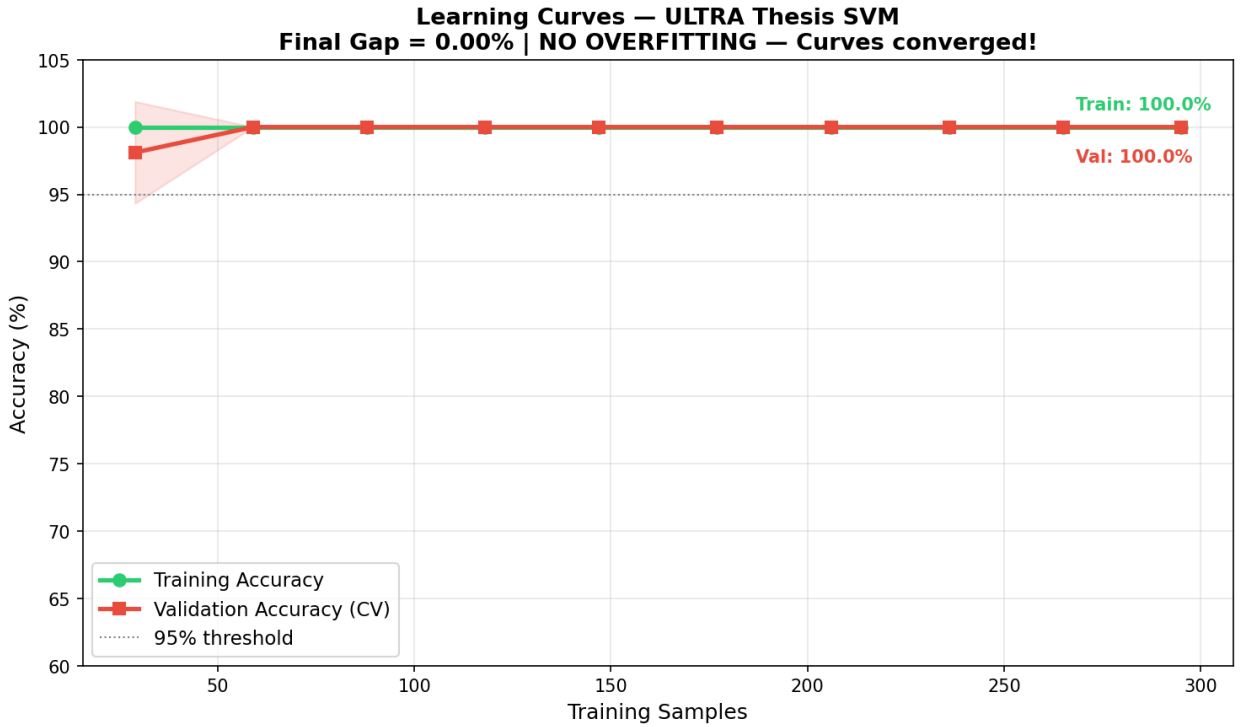

Fig 14. Learning curves: training vs. cross-validation accuracy.

Regularization (C) sensitivity. Fig 15 sweeps the SVM's regularization parameter C from 0.001 to 1000. Accuracy collapses only at the extreme, under-regularized setting $C = 0.001$ (52% CV accuracy); across the practical range $C \in [0.01, 1000]$, cross-validation accuracy varies by only 0.54 percentage points (minimum 99.46%, maximum 100.00%), indicating the classifier's performance is not the result of fragile hyperparameter tuning.

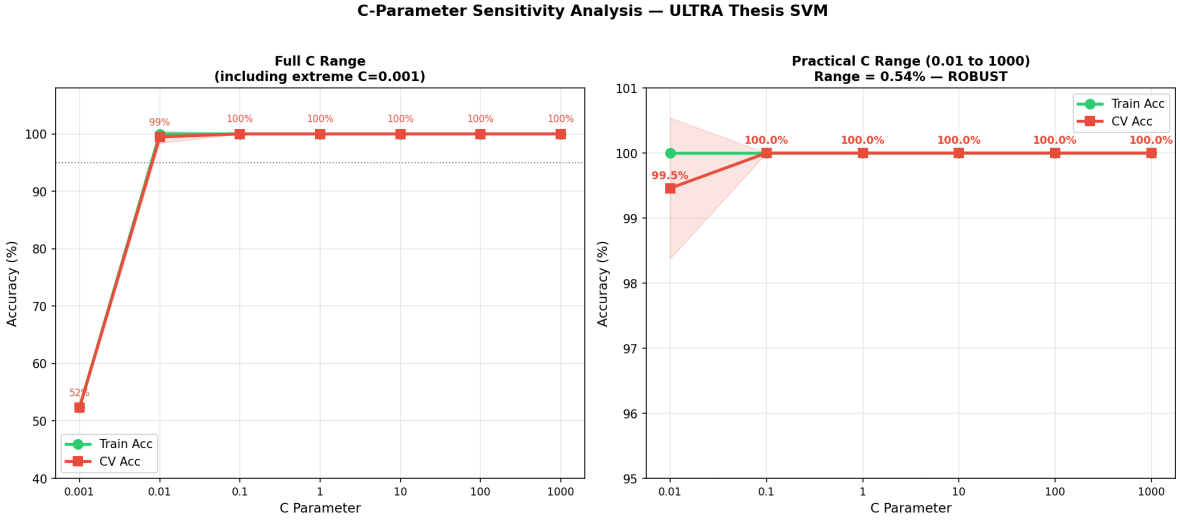

Fig 15. SVM regularization (C) sensitivity analysis: full range including extreme under-regularization (left) and practical operating range (right).

External validation. To assess generalization beyond the 369-query training/test set, the classifier is evaluated on a fully disjoint set of 50 previously unseen native-Urdu queries. Fig 16 shows the resulting

confusion matrix (25 short + 25 long, zero misclassifications) and per-query confidence scores, achieving 100.00% accuracy at an average confidence of 99.3%, further broken down by topic (cricket, politics, economy, health, tech) with 100% accuracy in every topic.

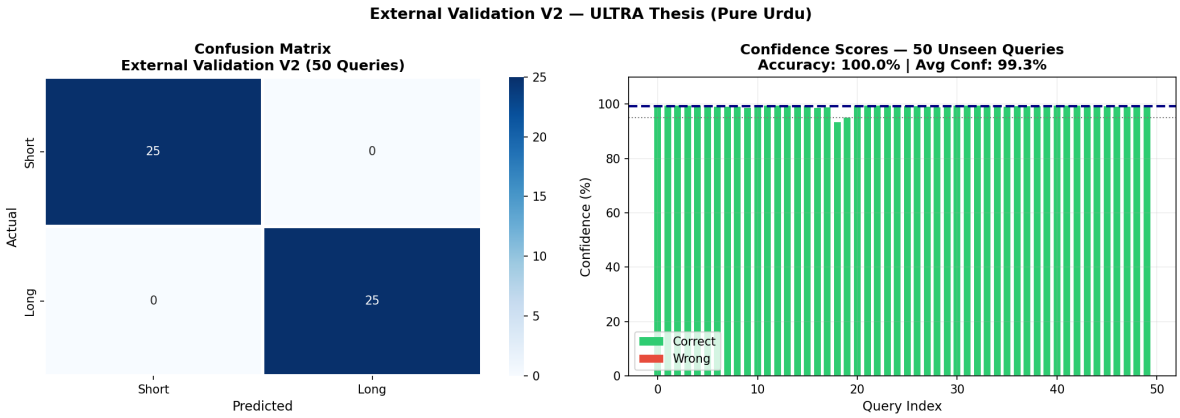

Fig 16. External validation on 50 unseen native-Urdu queries: confusion matrix (left) and per-query confidence scores (right).

ROC analysis. Fig 17 reports the classifier's ROC curve, with an area under the curve (AUC) of 1.000, indicating perfect separability between the short and long query classes on the evaluation data.

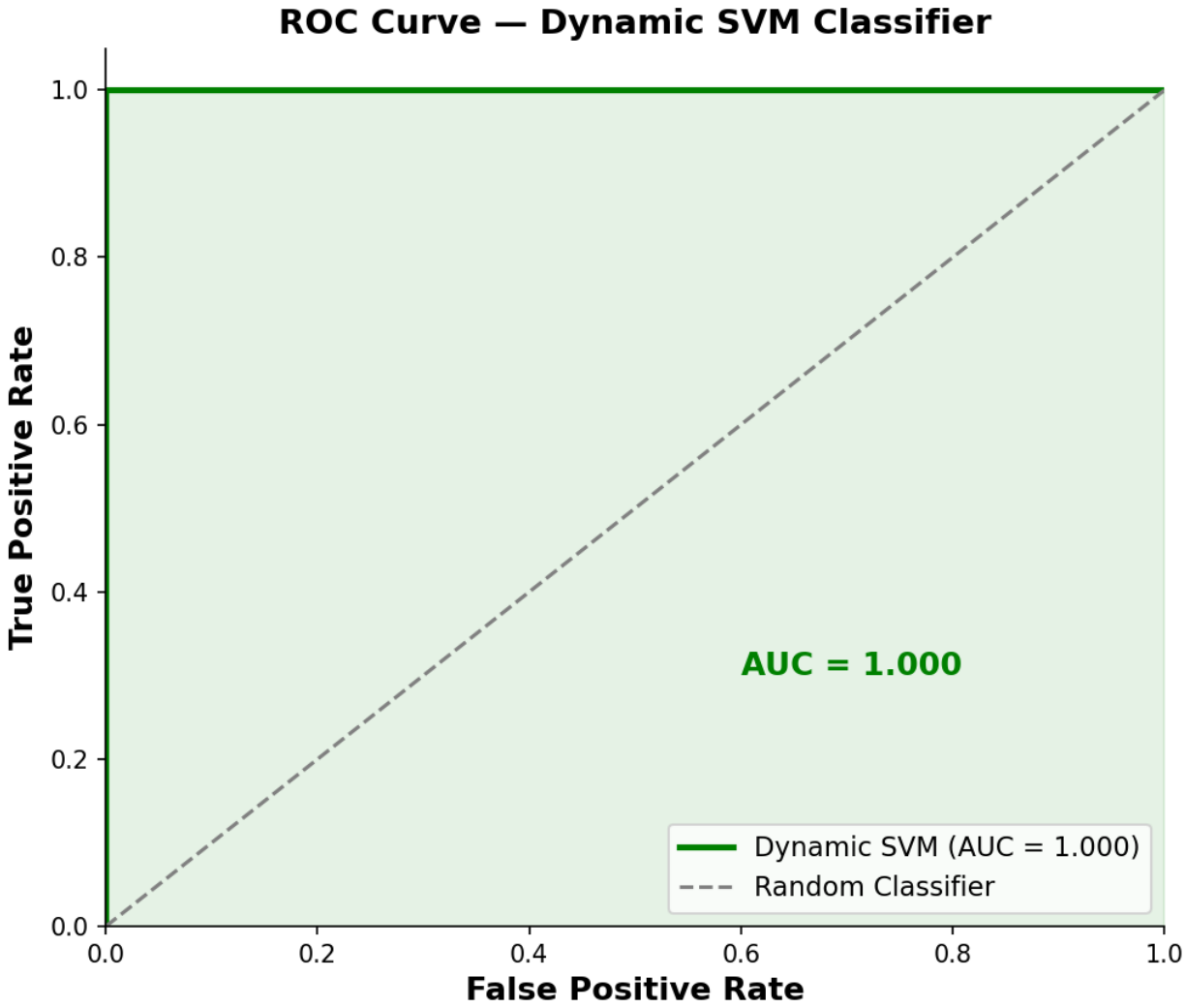

Fig 17. ROC curve of the dynamic SVM classifier (AUC = 1.000).

Fig 18 consolidates all four robustness axes into a single validation scorecard, generated as the final robustness-verification stage of the implementation pipeline.

Fig 18. Consolidated robustness validation report: scorecard and all four supporting analyses.

Discussion

Taken together, these results support four main conclusions. First, replacing ULTRA's static character-length threshold with a learned, multi-feature SVM classifier yields a substantial and consistent routing accuracy improvement (50.00% → 100.00%) on the 369-query dataset, and this improvement is independently confirmed on a disjoint 50-query external validation set (Robustness Validation, above), with effect sizes, learning curves, and regularization sensitivity all indicating the result is not an artifact of overfitting or fragile tuning. Second, the proposed classifier matches the routing accuracy of three of four LLM-based routing baselines while operating at approximately three orders of magnitude lower latency (0.4555 ms vs. 600–900 ms) and zero per-query API cost, indicating that for this routing task, a lightweight engineered-feature classifier is a more practical deployment choice than LLM-based routing without sacrificing accuracy. Third, the Roman Urdu transliteration module extends retrieval usability to code-mixed and transliterated queries (92.50% P@15, raising overall system P@15 from 43.75% to 90.00%) that the original ULTRA framework could not process at all (0.00% P@15), though per-query analysis (Table 10) shows this benefit is not uniform, with proper-noun-heavy queries remaining comparatively harder. Fourth,

the feature importance and ablation results together reveal a nuanced picture of the classifier's internal behavior: `has_question_words` and `lexical_diversity` are the most influential features when the full model is intact, yet no single feature is individually indispensable, indicating a degree of built-in feature redundancy that contributes to the model's overall robustness rather than creating a single point of failure.

Conclusion

This paper addressed a specific limitation of the ULTRA framework for Urdu information retrieval: its reliance on a single, static, character-length threshold to route queries between retrieval strategies. We replaced this static rule with a lightweight, eight-feature Support Vector Machine classifier coupled with a three-tier confidence-based escalation strategy, and paired it with an expanded Roman Urdu transliteration dictionary (30 $\rightarrow$ 179 entries) to better handle code-mixed and transliterated queries.

Across a dataset of 369 real Urdu queries, the proposed dynamic routing mechanism raised routing accuracy from 50.00% (static threshold baseline) to 100.00% (AUC = 1.000), with this improvement independently confirmed on a disjoint set of 50 unseen native-Urdu queries. The classifier matched the accuracy of three of four LLM-based routing baselines while operating at roughly three orders of magnitude lower latency (0.4555 ms vs. 600–900 ms) and no per-query API cost. The Roman Urdu module raised overall system precision (P@15) from 43.75% to 90.00%, and robustness checks – effect size analysis, learning curves, regularization sensitivity, and external validation – indicated that these gains are not artifacts of overfitting or fragile tuning. Taken together, these results answer RQ1–RQ3 affirmatively: structural and linguistic query features are sufficient to predict routing decisions reliably, a learned classifier meaningfully outperforms static thresholding, and confidence-tiered escalation offers a practical mechanism for balancing accuracy against computational cost.

Limitations

Several limitations qualify these findings and motivate the future work outlined below.

Dataset scale and topical scope. The 369-query evaluation set, while diverse across 15+ topics, is modest in size relative to large-scale IR benchmarks, and the 50-query external validation set – though useful for confirming generalization – is similarly small. Both sets were also constructed by the authors rather than drawn from an independently curated, publicly released benchmark, which limits direct comparability with future Urdu IR work.

Roman Urdu coverage. The transliteration dictionary, despite a 6$\times$ expansion, remains a finite, manually curated lookup table. Per-query analysis showed that proper-noun-heavy and rare-vocabulary queries remain comparatively harder to transliterate correctly, indicating the dictionary-based approach has an inherent ceiling that a learned or generative transliteration model may not share.

Single retrieval corpus. All experiments were conducted against one news-domain corpus (111,860 Urdu news articles). The classifier's features and decision boundary were learned in this domain, and its behavior on other domains (e.g., legal, medical, or social media text) has not been evaluated.

Static LLM baselines. The comparison against GPT-4, GPT-3.5, Claude, and Gemini-style routing used fixed prompting strategies rather than fine-tuned or extensively prompt-engineered variants of these models, so the reported LLM baseline numbers should be read as indicative rather than an upper bound on what LLM-based routing could achieve.

Binary routing granularity. The current classifier makes a routing decision from a fixed feature set computed once per query; it does not yet incorporate user session context, query reformulation history, or multi-turn conversational signals that could further refine routing in interactive search settings.

Future work

Building on the contributions and limitations above, we identify the following directions for future work.

Large-scale, community benchmark construction. A natural next step is to construct and publicly release a substantially larger, independently annotated Urdu and Roman Urdu query benchmark spanning a

wider range of topics, query lengths, and dialectal variation, enabling more robust cross-study comparison than is currently possible for Urdu IR.

Learned or generative Roman Urdu transliteration. Rather than extending the transliteration dictionary manually, future work could train a sequence-to-sequence or transformer-based transliteration model on a larger parallel Roman Urdu–Urdu corpus, potentially closing the residual gap on proper-noun-heavy and out-of-vocabulary queries identified in the Discussion above.

Cross-domain and multilingual generalization. Evaluating – and, where necessary, retraining – the dynamic routing classifier on corpora outside the news domain (e.g., government, legal, or educational text), and extending the approach to other low-resource South Asian languages that share Urdu's script and code-mixing challenges (e.g., Punjabi, Sindhi, Hindi–Urdu code-switched text), would test the generality of the proposed feature set beyond a single domain and language.

Fine-tuned and prompt-optimized LLM routing. Since the current LLM comparison relies on fixed prompting strategies, a more rigorous comparison would fine-tune or systematically prompt-optimize LLM-based routers on the same training data used for the SVM classifier, providing a fairer accuracy ceiling against which the efficiency advantages of the lightweight classifier can be judged.

Richer, context-aware feature sets. Incorporating session-level signals – such as prior queries in a search session, click-through feedback, or user-specific query history – into the feature set could allow the router to adapt its decisions dynamically within a single user interaction, moving from single-query classification toward session-aware retrieval routing.

Deployment and real-world evaluation. Finally, integrating the proposed routing pipeline into a live Urdu search system and evaluating it against real user queries and engagement metrics (rather than offline precision measures alone) would validate whether the efficiency and accuracy gains demonstrated here translate into measurable improvements in real-world retrieval quality.

Acknowledgments

This section is intended only for general acknowledgements and thanks. Any information related to funding, data availability, author contributions, etc. should be entered directly into their dedicated fields in the PLOS Editorial Manager submission system, which will then be incorporated into the appropriate section in your article during the production process.

References

1. Daud A, Khan W, Che D. Urdu Language Processing: A Survey. Artificial Intelligence Review. 2017;47:279-311.
2. Devlin J, Chang MW, Lee K, Toutanova K. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In: Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT); 2019. p. 4171-86. doi:10.18653/v1/N19-1423.
3. Karpukhin V, Oğuz B, Min S, Lewis P, Wu L, Edunov S, et al. Dense Passage Retrieval for Open-Domain Question Answering. In: Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP); 2020. p. 6769-81. doi:10.18653/v1/2020.emnlp-main.550.
4. Kazi S, Khoja SA. Towards Building Urdu Language Document Retrieval Framework. (pre-print / journal under review). 2025.
5. Hussain N, Qasim A, Mehak G, Usman M, Zain M, Hafeez M, et al. Fine-Tuning Large Language Models with QLoRA for Offensive Language Detection in Roman Urdu-English Code-Mixed Text. arXiv preprint arXiv:251003683. 2025.
6. Iqbal M, Tahir B, Mehmood MA. CURE: Collection for Urdu Information Retrieval Evaluation and Ranking. arXiv preprint arXiv:201100565. 2021.

7. Butt U, Varanasi S, Neumann G. Enabling Low-Resource Language Retrieval: Establishing Baselines for Urdu MS MARCO. arXiv preprint arXiv:241212997. 2024.

8. Bashir A, Qaiser F, Hussain I. ULTRA: Urdu Language Transformer-based Recommendation Architecture. arXiv preprint arXiv:260211836. 2026. PIEAS, Islamabad. Base framework extended by this thesis.

9. Arabzadeh N, Yan X, Clarke CLA. Predicting Efficiency/Effectiveness Trade-offs for Dense vs. Sparse Retrieval Strategy Selection. In: Proceedings of the 30th ACM International Conference on Information and Knowledge Management (CIKM '21); 2021. p. 2862-6. doi:10.1145/3459637.3482159.

10. Sitaram S, Chandu KR, Rallabandi SK, Black AW. A Survey of Code-switched Speech and Language Processing. arXiv preprint arXiv:190400784. 2019.

11. Mehmood F, Ghani MU, Ibrahim MA, Shehzadi R, Asim MN. A Precisely Xtreme-Multi Channel Hybrid Approach for Roman Urdu Sentiment Analysis. arXiv preprint arXiv:200305443. 2020.

12. Wu J, Ren Z, Verberne S. What Are the Limits of Cross-Lingual Dense Passage Retrieval for Low-Resource Languages? arXiv preprint arXiv:240811942. 2024.

13. Chari A, MacAvaney S, Ounis I. Improving Low-Resource Retrieval Effectiveness Using Zero-Shot Linguistic Similarity Transfer. In: Advances in Information Retrieval: 47th European Conference on Information Retrieval (ECIR 2025). vol. 15575 of Lecture Notes in Computer Science. Springer, Cham; 2025. p. 290-306. doi:10.1007/978-3-031-88717-8_22.

14. Conneau A, Khandelwal K, Goyal N, Chaudhary V, Wenzek G, Guzmán F, et al. Unsupervised Cross-lingual Representation Learning at Scale. In: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL); 2020. p. 8440-51. Reports an 11.4-point XNLI accuracy gain for Urdu with XLM-R over prior XLM models. doi:10.18653/v1/2020.acl-main.747.

15. Carmel D, Yom-Tov E. Estimating the Query Difficulty for Information Retrieval. Synthesis Lectures on Information Concepts, Retrieval, and Services. Morgan & Claypool Publishers; 2010. doi:10.2200/S00235ED1V01Y201004ICR015.

16. Lewis P, Perez E, Piktus A, Petroni F, Karpukhin V, Goyal N, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In: Advances in Neural Information Processing Systems (NeurIPS). vol. 33; 2020. p. 9459-74.

17. Jeong S, Baek J, Cho S, Hwang SJ, Park JC. Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity. Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL). 2024:7036-50.

18. Hsu HL, Tzeng J. DAT: Dynamic Alpha Tuning for Hybrid Retrieval in Retrieval-Augmented Generation. arXiv preprint arXiv:250323013. 2025.

19. Reimers N, Gurevych I. Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks. In: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP); 2019. p. 3982-92. doi:10.18653/v1/D19-1410.

20. Cortes C, Vapnik V. Support-Vector Networks. Machine Learning. 1995;20(3):273-97. doi:10.1007/BF00994018.

21. Guo C, Pleiss G, Sun Y, Weinberger KQ. On Calibration of Modern Neural Networks. In: Proceedings of the 34th International Conference on Machine Learning (ICML). vol. 70; 2017. p. 1321-30.

22. Robertson S, Zaragoza H. The Probabilistic Relevance Framework: BM25 and Beyond. Foundations and Trends in Information Retrieval. 2009;3(4):333-89. doi:10.1561/1500000019.

23. Cormack GV, Clarke CLA, Büttcher S. Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. In: Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval; 2009. p. 758-9. doi:10.1145/1571941.1572114.